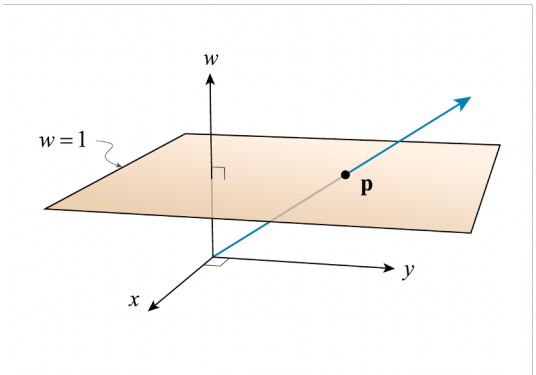
Geometric Algebra (GA)

Daniel Hug

2025-2026



Projective point. Source: „Foundations of Projective Geometric Algebra“, presentation Eric Lengyel, 05/2024

Contents

1 GA library **3**
1.1 Introduction to the ga library 3
1.2 Basics of the mathematical approach 9
1.3 Modelling motion 27
1.4 Physics modelling 46
1.4.1 Modelling force and torque 48
1.4.2 Modelling linear and angular momentum 51
1.4.3 Solving the Newton/Euler equations numerically . . . 60
1.4.4 Structure of inertia and inertia maps 68
1.5 Approaches with increasing expressivenes 78
1.5.1 Classic approach (vector algebra) 78
1.5.2 General observations for Geometric Algebra (GA) . . . 78
1.5.3 Euclidean Geometric Algebra (EGA) 78
1.5.4 Projective (Euclidean) Geometric Algebra (PGA) . . . 79
1.6 Glossary 80
1.7 Links to get started 81
1.8 Literature 82

1 GA library

1.1 Introduction to the ga library

The ga library is intended to make Geometric Algebra (GA) more accessible by providing functionality for numerical calculations required for simulation of physical systems.

It implements functionality for two- and three-dimensional spaces with Euclidean geometry. Currently, it provides data types and operations to handle the algebras of two- and three-dimensional Euclidean space ($EGA2D$, $EGA3D$) as well as for projective two- and three-dimensional Euclidean space ($PGA2DP$, $PGA3DP$). The latter two are modelled in a three- and four-dimensional space, respectively, since both include an additional projective dimension.

The library efficiently handles fully populated multivectors, even and odd-grade multivectors as well as their components like scalars, vectors, bivectors, trivectors, and pseudoscalars of the corresponding linear spaces. It also provides operations to work with these objects. It was designed with extensibility in mind and can be extended to provide functionality for handling additional algebras, e.g. spacetime-algebra (STA), with limited effort.

The main library is a header only library in the `ga` sub-folder of the library which can be used by including either

```
#include ga/ga_ega.hpp // and/or
#include ga/ga_pga.hpp
```

and by making the corresponding namespaces accessible by stating

```
using namespace hd::ga;      // and
using namespace hd::ga::ega; // either/or
using namespace hd::ga::pga;
```

in your application code. For a simple usage example please have a look either at `ga/ga_test/src/ga_ega_test.cpp` or at `ga/ga_test/src/ga_pga_test.cpp`.

At the heart of the library there are user defined types that model mathematical objects of the respective geometric algebra. They are realized using

type tagging in C++ in order to maximize reuse and to avoid code duplication. All types are defined in `ga/detail/type_t`.

To give an example for *EGA2D*, there are type aliases defined in the file `ga/detail/type_t/type2d.hpp` which provide types for scalars, vectors, pseudoscalars, even grade multivectors and full multivectors with the corresponding number of components. We have to distinguish between scalars and pseudoscalars at type level in order to allow for complement operations (defined later on) that can distinguish those types for each algebra. Following types are provided:

```
namespace hd::ga {

template <typename T> using Scalar2d
                            = Scalar_t<T,scalar2d_tag>;
template <typename T> using Vec2d = Vec2_t<T,vec2d_tag>;
template <typename T> using PScalar2d
                            = Scalar_t<T,pscalar2d_tag>;
template <typename T> using MVec2d_E = MVec2_t<T,mvec2d_e_tag>;
template <typename T> using MVec2d = MVec4_t<T,mvec2d_tag>;

} // namespace hd::ga
```

Based on these template aliases types like `Vec2d<T>` can be used directly. However, for increased user convenience additional template specializations have been defined in `ga/ga_usr_types.hpp` as

```
using scalar2d = Scalar2d<value_t>;
using vec2d = Vec2d<value_t>;
using pscalar2d = PScalar2d<value_t>;
using mvec2d_e = MVec2d_E<value_t>;
using mvec2d = MVec2d<value_t>;
```

where `value_t` is defined for the whole library in `ga/value_t.hpp`. Thus the user can simply select `value_t=double` (set as default value) or `float` for the library and the user application at one location. There is no need to use template parameters at all, if the user-defined types like `scalar2d,vec2d, pscalar2d,mvec2d_e`, and `mvec2d` are used for the application.

All algebras follow the same pattern. Just to give a second example for *PGA3DP* following user types are defined: `scalar3dp,vec3dp,bivec3dp, trivec3dp,pscalar3dp,mvec3dp_e,mvec3dp_u,` and `mvec3dp`, where we

additionally have defined types for bivectors, trivectors and uneven grade multivectors, as required to implement the algebra.

To work with the user defined algebraic objects geometric algebra operations using these types are defined in the ga library. You will find all operations that are required to work with linear vector spaces and operations specific to geometric algebra.

All product expressions for inner, outer, interior and geometric products as well as their regressive counterparts are generated by programs in the subfolder `ga/ga_prdxpr`. User input to configure product generation is provided in the header files of the corresponding algebra as well. The generated coefficient expressions are used in the source code to implement dot products, wedge products, commutator products, contractions, expansions, geometric products or their regressive counterparts, as well as sandwich products describing rotors and motors of the corresponding algebra. If certain products are required for your applicaton, but not yet implemented in the ga library, they can be easily generated within `ga/ga_prdxpr/src_prdxpr` as needed and added with minimal effort.

As mentioned before, we distinguish inner, interior, outer and geometric products.

- Inner ($a \cdot b$) and interior products, like contractions and expansions ($a \vee b^\star$, $a \wedge b^\star$), are grade reducing operations. The inner product combines arguments of the same type and grade, while the interior products involve duals of one of their arguments, combining and linking arguments of different grades. Inner products are linked to the metric of the modelled space.

- The outer product ($a \wedge b$) is a grade-increasing operation adding the grades of its arguments. Outer products, like the wedge product, connect subspaces represented by their operands and create new subspaces defined by the span of the subspaces of the operands. They establish a join-operation between the involved subspaces.

- The geometric product ($a \wedge\!\!\!\wedge b$) is the combination of both types of products in one expression. They can result in multivectors consisting of parts with different grades as result of the operation. The main advantage of the geometric product is its invertibility under certain circumstances.

- Regressive products ($a \circ b \equiv \underline{\overline{a} \cdot \overline{b}}$, $a \vee b \equiv \underline{\overline{a} \wedge \overline{b}}$, $rcmt(a,b) \equiv \underline{cmt(\overline{a},\overline{b})}$, $a \veebar b \equiv \underline{\overline{a} \barwedge \overline{b}}$) use complement operations on their arguments before and after applying the operation[1]. The regressive wedge product creates an intersection of the (sub-)spaces used by their arguments, thus effectively creating a meet-operation between the involved spaces.

- Sandwich products ($R \barwedge u \barwedge \widetilde{R}$ for EGA, $M \veebar u \veebar \underset{\sim}{M}$ for PGA) are used to implement orthogonal transformations of geometric objects. They realize rotations for EGA and general motions including rotation and translation for PGA by concatenating two consecutive reflections, used as primitives to realize the resulting transformations. This can be achieved algebraically by multiplying the object symmetrically from the left- and right-hand side with the appropriate unitized objects, we either call rotors R in EGA or motors M in PGA.

Following bi-argument operations $op(a,b) = a\ op\ b$ and their regressive variants $rop(a,b) = \underline{\overline{a}\ op\ \overline{b}}$ are implemented (with `op(a,b)` or `rop(a,b)` as the name of the implemented function). Using A vs. a implies $gr(A) \geq gr(a)$, where $gr(arg)$ returns the grade of the argument arg:

- Inner product, delivering a scalar: $a \cdot b$, `dot(a,b)`

- Outer product of a j- and a k-vector, delivering a $j+k$-vector in $span(a,b)$: $a \wedge b$, `wdg(a,b)`

- Geometric product: $a \barwedge b$, `operator*(a,b)`

  - For vectors a and b: $a \barwedge b = a \cdot b + a \wedge b$
  - For a vector a and a multivector B: $a \barwedge B = B \gg a + a \wedge B$
  - For a multivector A and a vector b: $A \barwedge b = b \ll A + A \wedge b$

- Commutator product (the asymmetric part of the geometric product): $cmt(a,b) \equiv \frac{1}{2}(a \barwedge b - b \barwedge a)$,

- Regressive commutator product (the asymmetric part of the regressive geometric product):
  $rcmt(a,b) \equiv \frac{1}{2}(\underline{\overline{a} \barwedge \overline{b} - \overline{b} \barwedge \overline{a}}) = \frac{1}{2}(a \veebar b - b \veebar a)$, `rcmt(a,b)`

- Regressive inner product, delivering a pseudoscalar: $a \circ b$, `rdot(a,b)` (for comparison of the dot product and the regressive dot product for $PGA3DP$ see table 1 at the end of section 1.2)

[1] for definition of the left and right complement operations marked with underbar and overbar, see next section

- Regressive outer product: $a \vee b$, `rwdg(a,b)`
- Regressive geometric product: $a \veebar b$, `rgpr(a,b)`
- Left (bulk) contraction: $a \rfloor B = a \ll B = a_\star \vee B$, `operator<<(a,B)`, which implements `left_bulk_contract(a,B)`
- Right (bulk) contraction: $B \lfloor a = B \gg a = B \vee a^\star$, `operator>>(B,a)`, which implements `right_bulk_contract(B,a)`
- Left weight contraction: $a_\star \vee B$, `left_weight_contract(a,B)`
- Right weight contraction: $B \vee a^\star$, `right_weight_contract(B,a)`
- Left bulk expansion: $A_\star \wedge b$, `left_bulk_expand(A,b)`
- Right bulk expansion: $a \wedge B^\star$, `right_bulk_expand(a,B)`
- Left weight expansion: $A_\star \wedge b$, `left_weight_expand(A,b)`
- Right bulk expansion: $a \wedge B^\star$, `right_weight_expand(a,B)`

where the items from the left contraction downwards are all considered interior products. They operate directly on the subspaces occupied by the objects. The product operation is performed after applying a dualization operation to one operand.

The product tables for all products are generated in `ga/ga_prdxpr`, and coefficient expressions for specific argument types like scalars, vector, bivectors, etc. can be generated based on those product tables using configuration options of `ga/ga_prdxpr`.

PGA explicitly means Projective Geometric Algebra in the context of the ga library. There is also an alternative approach called Plane-based Geometric Algebra (also abbreviated *PGA*) that is based on a space that is dual to the space modelled and used here. The alternative approach was introduced by [Gunn, 2020] and further promoted for applications in projective geometry by [Dorst and De Keninck, 2022]. That model uses dual representations of objects represented in the algebra (it uses $G^*(3,0,1)$, a space dual to $G(3,0,1)$). Planes are used instead of vectors to model everything, e.g. a point is modelled by three planes intersecting in a point. Both representations are mathematically equivalent, i.e. they are dual to each other. They are linked by the definition of complements and the interconnection between geometric and regressive geometric product, as defined above: $a \veebar b \equiv \underline{\overline{a} \wedge \overline{b}}$.

In the plane-based approach the geometric product is used to operate on complements and duals, which are defined as the primary objects used for the approach, while the approach introduced by Lengyel in [Lengyel, 2024b] favors the use of points, vectors, bivectors and planes plus the explicit use of regressive products and complements as well as duals for applying geometric operations to these objects. The latter approach unifies the use of the wedge product for join operations and of the regressive wedge product for meet operations for application in EGA and PGA. For the ga library we use $G(3, 0, 1)$.

1.2 Basics of the mathematical approach

The following is provided for Euclidean geometric algebra of two-dimensional space ($EGA2D$) using an orthormal basis $\boldsymbol{e}_1$, $\boldsymbol{e}_2$:

$$(\boldsymbol{e}_1)^2 = \boldsymbol{e}_1 \wedge \boldsymbol{e}_1 = \boldsymbol{e}_1 \cdot \boldsymbol{e}_1 = \boldsymbol{e}_1^2 = +1, \text{ and } \boldsymbol{e}_2^2 = +1$$

$$s_{2d} = s\,\mathbf{1} \tag{1a}$$

$$v_{2d} = v_1\boldsymbol{e}_1 + v_2\boldsymbol{e}_2 \tag{1b}$$

$$ps_{2d} = ps\,\boldsymbol{e}_{12} = ps\,\mathbb{1} \tag{1c}$$

equation (1a) is the scalar part s_{2d}, equation (1b) contains the vector part v_{2d}, and equation (1c) contains the pseudoscalar part ps_{2d}. The index $2d$ is omitted when clear from context. The four basis elements are $\{\mathbf{1}, \boldsymbol{e}_1, \boldsymbol{e}_2, \boldsymbol{e}_{12}\}$. Using this basis a multivector M of $2d$-space is defined as

$$M = s\,\mathbf{1} + v_1\boldsymbol{e}_1 + v_2\boldsymbol{e}_2 + ps\,\boldsymbol{e}_{12} \tag{2}$$

where equation (2) contains three parts: the scalar part $s\,\mathbf{1}$ (basis element is the scalar $\mathbf{1}$; if $\mathbf{1}$ is not shown, it is implicitly assumed for scalar values), the vector part v (basis elements $\boldsymbol{e}_1$ and $\boldsymbol{e}_2$) and the pseudoscalar part $ps\,\boldsymbol{e}_{12} = ps\,\mathbb{1}$ (basis element is $I_{2d} = \boldsymbol{e}_{12}$, which is sometimes written as $\mathbb{1}$ to show its character as pseudoscalar of this space. I_{2d} is a bivector in two-dimensional Euclidean space).

For Euclidean geometric algebra of three-dimensional space ($EGA3D$) using an orthonormal basis $\boldsymbol{e}_1$, $\boldsymbol{e}_2$, $\boldsymbol{e}_3$ there are:

$$(\boldsymbol{e}_1)^2 = \boldsymbol{e}_1 \wedge \boldsymbol{e}_1 = \boldsymbol{e}_1 \cdot \boldsymbol{e}_1 = \boldsymbol{e}_1^2 = +1, \boldsymbol{e}_2^2 = +1, \text{ and } \boldsymbol{e}_3^2 = +1$$

$$s_{3d} = s\,\mathbf{1} \tag{3a}$$

$$v_{3d} = v_1\boldsymbol{e}_1 + v_2\boldsymbol{e}_2 + v_3\boldsymbol{e}_3 \tag{3b}$$

$$B_{3d} = b_1\boldsymbol{e}_{23} + b_2\boldsymbol{e}_{31} + b_3\boldsymbol{e}_{12} \tag{3c}$$

$$ps_{3d} = ps\,\boldsymbol{e}_{123} = ps\,\mathbb{1} \tag{3d}$$

equation (3a) is the scalar part s_{3d}, equation (3b) is the vector part v_{3d}, equation (3c) is the bivector part B_{3d}, and equation (3d) is the pseudoscalar part ps_{3d}. The index $3d$ is omitted when clear from context. Compared to the $2d$-case it is obvious, that all parts depend on context, specifically on the dimensionality of the modelled space, and thus need to be defined and treated accordingly.[2] The basis elements are $\{\mathbf{1}, \boldsymbol{e}_1, \boldsymbol{e}_2, \boldsymbol{e}_3, \boldsymbol{e}_{23}, \boldsymbol{e}_{31}, \boldsymbol{e}_{12}, \boldsymbol{e}_{123}\}$.

[2] *hint*: Since C++ is a statically typed language, those types need to be well-defined and distinguishable from each other. This is especially important for the different scalar and pseudoscalar types. The ga library uses type tagging extensively for that purpose.

Using this basis a multivector M of $3d$-space is defined as

$$M = s\,\mathbf{1} + v_1\boldsymbol{e}_1 + v_2\boldsymbol{e}_2 + v_3\boldsymbol{e}_3 + b_1\boldsymbol{e}_{23} + b_2\boldsymbol{e}_{31} + b_3\boldsymbol{e}_{12} + ps\,\boldsymbol{e}_{123} \tag{4}$$

where equation (4) contains four parts: the scalar component $s\,\mathbf{1}$ (basis element is the scalar $\mathbf{1}$), the vector part v (basis elements $\boldsymbol{e}_1$, $\boldsymbol{e}_2$ and $\boldsymbol{e}_3$), the bivector part B (basis elements $\boldsymbol{e}_{23}$, $\boldsymbol{e}_{31}$ and $\boldsymbol{e}_{12}$) and the pseudoscalar part $ps\,\boldsymbol{e}_{123} = ps\,\mathbb{1}$ (basis element is $I_{3d} = \boldsymbol{e}_{123}$, which is sometimes written as $\mathbb{1}$ to show its character as pseudoscalar of this space. I_{3d} is a trivector in three-dimensional Euclidean space).

Projective geometric algebra of $2D$ Euclidean space ($PGA2DP$) is modelled by embedding it in a three-dimensional space. Using an orthonormal basis $\boldsymbol{e}_1$, $\boldsymbol{e}_2$, $\boldsymbol{e}_3$ there are following basis vectors:

$$(\boldsymbol{e}_1)^2 = \boldsymbol{e}_1 \wedge \boldsymbol{e}_1 = \boldsymbol{e}_1 \cdot \boldsymbol{e}_1 = \boldsymbol{e}_1^2 = +1, \boldsymbol{e}_2^2 = +1, \text{ and } \boldsymbol{e}_3^2 = 0$$

$$s_{2dp} = s\,\mathbf{1} \tag{5a}$$

$$v_{2dp} = v_1\boldsymbol{e}_1 + v_2\boldsymbol{e}_2 + v_3\boldsymbol{e}_3 \tag{5b}$$

$$B_{2dp} = b_1\boldsymbol{e}_{31} + b_2\boldsymbol{e}_{32} + b_3\boldsymbol{e}_{12} \tag{5c}$$

$$ps_{2dp} = ps\,\boldsymbol{e}_{321} = ps\,\mathbb{1} \tag{5d}$$

The most obvious difference is the basis vector $\boldsymbol{e}_3$ that squares to zero and the pseudoscalar of that space that is chosen to be $\boldsymbol{e}_{321}$. The trivector $\boldsymbol{e}_{321}$ squares to 0, because of it's component $\boldsymbol{e}_3$ contained as a factor. Equation (5a) is the scalar part s_{2dp}, equation (5b) is the vector part v_{2dp}, equation (5c) is the bivector part B_{2dp}[3], and equation (5d) is the pseudoscalar part ps_{2dp}. The index $2dp$ is omitted when clear from context. The eight basis elements are $\{\mathbf{1}, \boldsymbol{e}_1, \boldsymbol{e}_2, \boldsymbol{e}_3, \boldsymbol{e}_{31}, \boldsymbol{e}_{32}, \boldsymbol{e}_{12}, \boldsymbol{e}_{321}\}$. Using this basis a multivector M of two-dimensional projective space ($2dp$) is defined as

$$M = s\,\mathbf{1} + v_1\boldsymbol{e}_1 + v_2\boldsymbol{e}_2 + v_3\boldsymbol{e}_3 + b_1\boldsymbol{e}_{31} + b_2\boldsymbol{e}_{32} + b_3\boldsymbol{e}_{12} + ps\,\boldsymbol{e}_{321} \tag{6}$$

where equation (6) contains four parts: the scalar component $s\,\mathbf{1}$ (basis element is the scalar $\mathbf{1}$), the vector part v (basis elements $\boldsymbol{e}_1$, $\boldsymbol{e}_2$ and $\boldsymbol{e}_3$), the bivector part B (basis elements $\boldsymbol{e}_{23}$, $\boldsymbol{e}_{31}$ and $\boldsymbol{e}_{12}$) and the pseudoscalar part $ps\,\boldsymbol{e}_{321} = ps\,\mathbb{1}$ (basis element is $I_{2dp} = \boldsymbol{e}_{321}$, which is sometimes written as $\mathbb{1}$ to show its character as pseudoscalar of this space. It is a trivector in two-dimensional Euclidean projective space that squares to zero).

[3] Here we deviate from Lengyel's approach chosen in [Lengyel, 2024b]. Since we want to model physics, it is important to generate unified handling for two- and three-dimensional cases. The chosen sequence of basis bivectors assures that lines, which are used to model forces and moments later on, have their directions encoded in a unified way.

In $2D$ PGA a projective point Q is modelled as a vector

$$Q_{pga2dp} = \underbrace{q_x \boldsymbol{e}_1 + q_y \boldsymbol{e}_1}_{position} + \underbrace{q_z \boldsymbol{e}_3}_{weight} \tag{7}$$

where the position is encoded in the bulk part[4] of the point, and the weight is the projective component q_z that must be unitized (=scaled to 1.0) in order to project the point into the Euclidean space.

A line L is modelled as a bivector

$$L_{pga2dp} = \underbrace{l_x \boldsymbol{e}_{31} + l_y \boldsymbol{e}_{32}}_{direction} + \underbrace{l_z \boldsymbol{e}_{12}}_{position} \tag{8}$$

where the weight part encodes the two-dimensional direction vector (l_x, l_y) of the line and the bulk part encodes the position of the line as distance to the origin $O_{2dp} = (0, 0, 1)$. When $l_z = 0$ the line passes through the origin.

Projective geometric algebra of $3D$ Euclidean space ($PGA3DP$) is modelled by embedding it in a four-dimensional space. Using an orthonormal basis $\boldsymbol{e}_1$, $\boldsymbol{e}_2$, $\boldsymbol{e}_3$, $\boldsymbol{e}_4$ there are following basis vectors:

$$(\boldsymbol{e}_1)^2 = \boldsymbol{e}_1 \wedge \boldsymbol{e}_1 = \boldsymbol{e}_1 \cdot \boldsymbol{e}_1 = \boldsymbol{e}_1^2 = +1, \boldsymbol{e}_2^2 = +1, \boldsymbol{e}_3^2 = +1, \text{ and } \boldsymbol{e}_4^2 = 0$$

$$s_{3dp} = s\,\mathbf{1} \tag{9a}$$

$$v_{3dp} = v_1 \boldsymbol{e}_1 + v_2 \boldsymbol{e}_2 + v_3 \boldsymbol{e}_3 + v_4 \boldsymbol{e}_4 \tag{9b}$$

$$B_{3dp} = b_1 \boldsymbol{e}_{41} + b_2 \boldsymbol{e}_{42} + b_3 \boldsymbol{e}_{43} + b_4 \boldsymbol{e}_{23} + b_5 \boldsymbol{e}_{31} + b_6 \boldsymbol{e}_{12} \tag{9c}$$

$$t_{3dp} = t_1 \boldsymbol{e}_{423} + t_2 \boldsymbol{e}_{431} + t_3 \boldsymbol{e}_{412} + t_4 \boldsymbol{e}_{321} \tag{9d}$$

$$ps_{3dp} = ps\,\boldsymbol{e}_{1234} = ps\,\mathbb{1} \tag{9e}$$

Now we have a basis vector $\boldsymbol{e}_4$ that squares to zero. The bivector has six components, and there is a trivector t. The pseudoscalar $\boldsymbol{e}_{1234}$ is a quadvector that squares to 0, because of it's component $\boldsymbol{e}_4$ contained as a factor. Equation (9a) is the scalar part s_{3dp}, equation (9b) is the vector part v_{3dp}, equation (9c) is the bivector part B_{3dp}, equation (9d) is the trivector part t_{3dp}, and equation (9e) is the pseudoscalar part ps_{3dp}. The index $3dp$ is omitted when clear from context. The sixteen basis elements of $PGA3DP$ are: $\{\mathbf{1}, \boldsymbol{e}_1, \boldsymbol{e}_2, \boldsymbol{e}_3, \boldsymbol{e}_4, \boldsymbol{e}_{41}, \boldsymbol{e}_{42}, \boldsymbol{e}_{43}, \boldsymbol{e}_{23}, \boldsymbol{e}_{31}, \boldsymbol{e}_{12}, \boldsymbol{e}_{423}, \boldsymbol{e}_{431}, \boldsymbol{e}_{412}, \boldsymbol{e}_{321}, \boldsymbol{e}_{1234}\}$. A multi-

[4] Bulk and weight parts are defined further below in this section.

vector M of three-dimensional projective space ($3dp$) is defined as

$$\begin{aligned} M = s\,\mathbf{1} & \\ & + v_1\boldsymbol{e}_1 + v_2\boldsymbol{e}_2 + v_3\boldsymbol{e}_3 + v_4\boldsymbol{e}_4 \\ & + b_1\boldsymbol{e}_{41} + b_2\boldsymbol{e}_{42} + b_3\boldsymbol{e}_{43} + b_4\boldsymbol{e}_{23} + b_5\boldsymbol{e}_{31} + b_6\boldsymbol{e}_{12} \\ & + t_1\boldsymbol{e}_{423} + t_2\boldsymbol{e}_{431} + t_3\boldsymbol{e}_{412} + t_4\boldsymbol{e}_{321} \\ & + ps\,\boldsymbol{e}_{1234} \end{aligned} \tag{10}$$

where equation (10) contains five parts: the scalar component $s\,\mathbf{1}$ (basis element is the scalar $\mathbf{1}$), the vector part v (basis elements $\boldsymbol{e}_1$, $\boldsymbol{e}_2$, $\boldsymbol{e}_3$, $\boldsymbol{e}_4$), the bivector part B (basis elements $\boldsymbol{e}_{41}$, $\boldsymbol{e}_{42}$, $\boldsymbol{e}_{43}$, $\boldsymbol{e}_{23}$, $\boldsymbol{e}_{31}$, $\boldsymbol{e}_{12}$), the trivector part t (basis elements $\boldsymbol{e}_{423}$, $\boldsymbol{e}_{431}$, $\boldsymbol{e}_{412}$, $\boldsymbol{e}_{321}$) and the pseudoscalar part $ps\,\boldsymbol{e}_{1234} = ps\,\mathbb{1}$ (basis element is $I_{3dp} = \boldsymbol{e}_{1234}$, which is sometimes written as $\mathbb{1}$ to show its character as pseudoscalar of this space. It is a quadvector in three-dimensional Euclidean projective space that squares to zero).

In $3D\ PGA$ a projective point Q is modelled as a vector

$$Q_{pga3dp} = \underbrace{q_x\boldsymbol{e}_1 + q_y\boldsymbol{e}_1 + q_z\boldsymbol{e}_1}_{position} + \underbrace{q_w\boldsymbol{e}_4}_{weight} \tag{11}$$

where the position is encoded in the bulk part of the point, and the weight is the projective component q_w that must be unitized (=scaled to 1.0) in order to project the point into the Euclidean space.

A line L is modelled as a bivector

$$L_{pga3dp} = \underbrace{l_{vx}\boldsymbol{e}_{41} + l_{vy}\boldsymbol{e}_{42} + l_{vz}\boldsymbol{e}_{43}}_{direction} + \underbrace{l_{mx}\boldsymbol{e}_{23} + l_{my}\boldsymbol{e}_{31} + l_{mz}\boldsymbol{e}_{12}}_{moment} \tag{12}$$

where the weight part encodes the three-dimensional direction vector of the line $l_v = (l_{vx}, l_{vy}, l_{vz})$, and the bulk part encodes the moment of the line $l_m = (l_{mx}, l_{my}, l_{mz})$ when interpreted as a vector. Both are encoded in the bivector object that captures all parameters needed to model the line.

Additionally, there are planes T that are modelled as trivectors

$$T_{pga3dp} = \underbrace{t_x\boldsymbol{e}_{423} + t_y\boldsymbol{e}_{431} + t_z\boldsymbol{e}_{412}}_{normal} + \underbrace{t_w\boldsymbol{e}_{312}}_{position} \tag{13}$$

where the first three components represent the normal vector of the plane $n = (t_x, t_y, t_z)$ and the fourth component t_w corresponds to the signed distance to the origin $O_{3dp} = (0, 0, 0, 1)$.

It is important to note that it is seldomly required to use a full multivector in the respective algebra. For most applications it is sufficient to use a blade or k-vector exclusively, which represents one grade of the multivector, e.g. a vector or a bivector. This reduces the effort in terms of memory and computational requirements significantly – typically reduced to a level that is required anyway to solve the underlying application problem.

The complement operation used in the library is defined in [Lengyel, 2024b]. It is uniquely determined with respect to the outer product (*not* with respect to the geometric product, as commonly used for dualization by many autors working with GA). This is ensures consistent signs of expressions, operators, complements and duals. The left complement $\underline{u}$ and the right complement $\overline{u}$ are defined as

$$\underline{u} \wedge u = I_n \tag{14a}$$
$$u \wedge \overline{u} = I_n \tag{14b}$$

with $I_n = \boldsymbol{e}_1 \wedge \boldsymbol{e}_2 \wedge \ldots \wedge \boldsymbol{e}_n$ as the pseudoscalar of the n-dimensional algebra, and the wedge symbol ($\wedge$) as the operator symbol of the outer product. The effect of the complement is to turn basis elements not contained in the object u into the constituting basis elements of the complement of the object $\underline{u}$ or $\overline{u}$. Connecting the object and its complement with the wedge products creates an object that fills the available space in the sense of requiring all basis elements of the space to represent it. The full space itself is represented by the pseudoscalar of that space, since the pseudoscalar I_n is formed by the product of all n basis vectors of the space. An implication of the definition is that the complement ($\underline{u}$ or $\overline{u}$) is orthogonal to u, since we exclusively use orthogonal basis vectors to model our space.

The complement is defined as given above in equations (14) for any basis element. It is extended to full multivectors A and B by linearity as

$$\overline{sA} = s\overline{A}, \quad \overline{A+B} = \overline{A} + \overline{B} \tag{15a}$$
$$\underline{sA} = s\underline{A}, \quad \underline{A+B} = \underline{A} + \underline{B} \tag{15b}$$

where s is a scalar. The left and right complements fulfill

$$\overline{\underline{u}} = u \tag{16}$$

thus the left and right complement operations are inverses to each other. This is valid independent of the sequence of application, independent of the grade of the argument a, and independent of the dimension of the space

modelled in the algebra. In spaces of odd dimension, where we only have one complement, applying the complement twice of course returns the original object as well.

However, applying the same complement complement operation several times in spaces of even dimension does not ensure that you get back the original object. The result is dependent on the grade of the basis vector and the dimension n of the space. When applying the same complement twice we get

$$\overline{\overline{u}} = \underline{\underline{u}} = (-1)^{gr(u)(n-gr(u))}u \tag{17}$$

i.e. this involves grade-dependent sign changes. So it is favorable to use left and right complements as inverses to each other.

Based on the given complement definition it is possible to define regressive products using rules analog to deMorgan's rules in Boolean algebra as follows (definitions can be found in [Lengyel, 2024b] and in [Browne, 2012]):

$$a \vee b \equiv \underline{\overline{a} \wedge \overline{b}} \tag{18a}$$

$$a \vee b \equiv \overline{\underline{a} \wedge \underline{b}} \tag{18b}$$

The same complement is applied to both arguments first, the resulting expressions are combined with the wedge product, and the inverse complement operation is used to transform the outcome into the final result, the regressive product. The inverted wedge symbol ($\vee$) is used for the regressive wedge product. Both definitions (18a) and (18b) are equivalent. Equation (18a) is used to derive expressions for regressive products in `ga/ga_prdxpr`.

There are some additional formulas for complements which are useful for working with GA expressions and thus are provided here for reference. Taking the right and left complements of (18a) and (18b) we end up with

$$\overline{a \vee b} = \overline{\underline{\overline{a} \wedge \overline{b}}} = \overline{a} \wedge \overline{b} \tag{19a}$$

$$\underline{a \vee b} = \underline{\overline{\underline{a} \wedge \underline{b}}} = \underline{a} \wedge \underline{b} \tag{19b}$$

Substituting $a = \underline{u}$ and $b = \underline{v}$ into (19a) and $a = \overline{u}$ and $b = \overline{v}$ into (19b) and exchanging the left and right hand sides of the equations we end up with

$$u \wedge v = \overline{\underline{u} \vee \underline{v}} \tag{20a}$$

$$u \wedge v = \underline{\overline{u} \vee \overline{v}} \tag{20b}$$

which shows that the outer and the regressive outer product can be expressed by each other.

The complement operation is used to define a unique dualization operation that works for cases with non-degenerate as well as for cases with degenerate metric, like in projective geometric algebra PGA. This is the main advantage compared to the usual prodedure of dualization by multiplying the object with the pseudoscalar (or its inverse) using the geometric product, since the latter breaks down for spaces with degenerate metrics. For an overview on all complements in PGA see tables 2 and 6 at the end of this section.

The metric is defined by the dot product of the basis vectors as

$$\mathfrak{g}_{ij} \equiv \boldsymbol{e}_i \cdot \boldsymbol{e}_j \tag{21}$$

It is a square matrix that is the identity matrix I_n for EGA, since all basis vectors square to $\boldsymbol{e}_i \cdot \boldsymbol{e}_i = +1$, and $\boldsymbol{e}_i \cdot \boldsymbol{e}_j = 0$ for $i \neq j$. It is invertible in this case. However, for PGA one basis vector squares to zero, and thus there is one zero on the main diagonal of the corresponding matrix of the metric, which is therefore not invertible (its determinant is zero). It is called degenerate for that reason.

The extended metric $\mathbf{G}$ is a so-called exomorphism matrix that fulfills

$$\mathbf{G}(A \wedge B) = \mathbf{G}(A) \wedge \mathbf{G}(B) \tag{22}$$

for any multivector A and B (see [Lengyel, 2024b], pp. 61ff.).
Regarding $\mathbf{G}(u) = \mathbf{G}u$ as a unary operation on u we can define the corresponding regressive operation by first taking the complement of the argument and then applying the opposed complement on the result. This defines the regressive metric $\mathbb{G}$ (named anti-metric by Lengyel). It is defined as

$$\mathbb{G}u \equiv \underline{\mathbf{G}\overline{u}} = \overline{\mathbf{G}\underline{u}} \tag{23}$$

$\mathbf{G}$ and $\mathbb{G}$ are related by

$$\mathbf{G}\mathbb{G} = det(\mathfrak{g})I_n \tag{24}$$

Thus, if all basis vectors square to ± 1, we have $\mathbf{G}\mathbb{G} = \pm I_n$, and both are invertible. However, if one basis vector squares to zero, we get $\mathbf{G}\mathbb{G} = 0$, which means that none of them is invertible, i.e. we have a degenerate metric.

For $\mathbb{G}$ there is the relation

$$\mathbb{G}(A \vee B) = \mathbb{G}(A) \vee \mathbb{G}(B) \tag{25}$$

that is complementary to equation (22).

In PGA the product $\mathbf{G}u$ defines the bulk of of an object u. It is defined as

$$u_\bullet = \mathbf{G}u \tag{26}$$

The bulk only contains components that *do not* contain the projective dimension ($\boldsymbol{e}_3$ for $PGA2DP$ or $\boldsymbol{e}_4$ for $PGA3DP$) as a factor. It contains positional information on u relative to the origin. Zero bulk means that u contains the origin.

In PGA the product $\mathbb{G}u$ defines the weight of of an object u. It is defined as

$$u_\circ = \mathbb{G}u \tag{27}$$

The weight selects components that *do* contain the projective dimension ($\boldsymbol{e}_3$ for $PGA2DP$ or $\boldsymbol{e}_4$ for $PGA3DP$) as a factor. It contains directional information on u, i.e. its attitude and direction independent of its position. Zero weight means that u is contained in the horizon, i.e. it is infinitely far away.

This effective split for each object into bulk and weight in projective algebras can be written as

$$u = u_\bullet + u_\circ \tag{28}$$

This split is also relevant for the corresponding norms

$$\|u\| = \|u\|_\bullet + \|u\|_\circ = \sqrt{A \cdot A} + \sqrt{A \circ A} \tag{29}$$

where the left hand side is called the geometric norm and the right hand side consists of the sums of the bulk and weight norms. The bulk norm returns a scalar value of the corresponding algebra, whereas the weight norm provides a pseudoscalar value of the respective algebra. Consequently, the geometric norm is an object consisting of two distinct parts, defined here as a dual number in the corresponding algebra. When we apply unitization to geometric objects we devide the object in such a way that the magnitude of the weight norm becomes a value of one. Both, bulk and weight norms, have a positive argument under the square root due to the definition of the inner product and the regressive inner product.

Using the metric and the right complement, the right bulk dual $A^\star$ is defined as

$$A^\star = \overline{\mathbf{G}A} = \overline{A_\bullet} \tag{30}$$

where A is an arbitrary multivector, $\mathbf{G}$ the extended metric and $\mathbf{G}A$ a matrix-vector-product between them. $A^\star$ is also called the Hodge dual. The multiplication with the metric happens before the application of the complement operation.

There is also a corresponding left bulk dual $A_\star$ operation using the metric and the left complement. It is defined as

$$A_\star = \underline{\mathbf{G}A} = \underline{A_\bullet} \tag{31}$$

The Hodge star (symbolized by a filled star $\star$) is used as dualization operator in both cases. The lower or upper position symbolizes the left and right application. The difference between the hodge dual and the complement is in both cases that multiplying the multivector with the extended metric happens *before* taking the corresponding complement. However, this does not lead to different results in spaces where the metric is the identity matrix. It only yields different results in spaces with a more involved metric, e.g. in projective spaces with a non-invertible (degenerate) metric as in PGA.

The effect of applying the metric before taking the complement is equivalent to applying the complement operation on the bulk part of the object u exclusively, and not on the full object u. Of course, the dualization can be applied to the weight part as well. This operation (symbolized by an unfilled star "$\star$") uses the regressive extended metric and is defined for the right weight dual $A^\star$ using the right complement as

$$A^\star = \overline{\mathbb{G}A} = \overline{A_\circ} \tag{32}$$

for an arbitrary multivector A. The left weight dual $A_\star$ is defined as

$$A_\star = \underline{\mathbb{G}A} = \underline{A_\circ} \tag{33}$$

using the left complement on the weight part of A.

Relations to the (regressive) geometric product product are given by

$$A^\star = \widetilde{A} \wedge I_n = \widetilde{A} \wedge \mathbb{1} \tag{34a}$$
$$A_\star = I_n \wedge \widetilde{A} = \mathbb{1} \wedge \widetilde{A} \tag{34b}$$
$$A^\star = \underset{\sim}{A} \vee \mathbf{1} \tag{34c}$$
$$A_\star = \mathbf{1} \vee \underset{\sim}{A} \tag{34d}$$

with $I_n = \boldsymbol{e}_1 \wedge \boldsymbol{e}_2 \wedge \ldots \wedge \boldsymbol{e}_n = \mathbb{1}$ as the pseudoscalar of the n-dimensional space. $\widetilde{A}$ is the reversion operation, and $\underset{\sim}{A}$ is the regressive reversion operation

applied to A (e.g. with $A = \boldsymbol{e}_1 \wedge \boldsymbol{e}_2$ the reverse is $\widetilde{A} = \boldsymbol{e}_2 \wedge \boldsymbol{e}_1$), and "$\wedge\!\!\!\!\wedge$" and "$\vee\!\!\!\!\vee$" are the symbols for the geometric and regressive geometric products. As the complement, the reverse is defined for any basis element. It is extended to full multivectors A and B by linearity as

$$\widetilde{sA} = s\widetilde{A}, \quad \widetilde{A+B} = \widetilde{A} + \widetilde{B}, \quad \widetilde{A \wedge\!\!\!\!\wedge B} = \widetilde{B} \wedge\!\!\!\!\wedge \widetilde{A} \tag{35a}$$

$$\underset{\sim}{sA} = s\underset{\sim}{A}, \quad \underset{\sim}{A+B} = \underset{\sim}{A} + \underset{\sim}{B}, \quad \underset{\sim}{A \vee\!\!\!\!\vee B} = \underset{\sim}{B} \vee\!\!\!\!\vee \underset{\sim}{A} \tag{35b}$$

$$\widetilde{\widetilde{A}} = A, \quad \text{and} \quad \underset{\sim}{\underset{\sim}{A}} = A \tag{35c}$$

where s is a scalar. The reverse and regressive reverse are their own inverses, i.e. applying the same reverse twice returns the original object.

For an overview on all bulk and weight duals in PGA see tables 3 and 7 at the end of this section. The reverses can be found in tables 4 and 8 at the end of the section as well.

Following identities link the expansions and contractions and thus the outer and inner products:

$$(A \wedge B)^{\star} = A^{\star} \vee B^{\star} \tag{36a}$$

$$A \wedge B^{\star} = (A \cdot B) \wedge \mathbb{1}, \quad \text{when } \texttt{gr(A)} = \texttt{gr(B)} \tag{36b}$$

$$A \wedge B^{\star\!\!\!\star} = A \circ B, \quad \text{when } \texttt{gr(A)} = \texttt{gr(B)} \tag{36c}$$

$$(A \vee B)^{\star\!\!\!\star} = A^{\star\!\!\!\star} \wedge B^{\star\!\!\!\star} \tag{36d}$$

$$A \vee B^{\star} = A \cdot B, \quad \text{when } \texttt{gr(A)} = \texttt{gr(B)} \tag{36e}$$

$$A \vee B^{\star\!\!\!\star} = (A \circ B) \vee \mathbf{1}, \quad \text{when } \texttt{gr(A)} = \texttt{gr(B)} \tag{36f}$$

The influence of the metric for the dualization operation can be seen in its effect for the different algebras which also depends on the dimension of the modelled space:

- $EGA2D$: $A_{\star} = \texttt{left_dual(A)}$, $A^{\star} = \texttt{right_dual(A)}$
- $EGA3D$: $A_{\star} = \texttt{dual(A)}$, $A^{\star} = \texttt{dual(A)}$
- $PGA2DP$: $A_{\star} = \texttt{bulk_dual(A)}$, $A^{\star} = \texttt{bulk_dual(A)}$
- $PGA2DP$: $A_{\star\!\!\!\star} = \texttt{weight_dual(A)}$, $A^{\star\!\!\!\star} = \texttt{weight_dual(A)}$
- $PGA3DP$: $A_{\star} = \texttt{left_bulk_dual(A)}$, $A^{\star} = \texttt{right_bulk_dual(A)}$
- $PGA3DP$: $A_{\star\!\!\!\star} = \texttt{left_weight_dual(A)}$, $A^{\star\!\!\!\star} = \texttt{right_weight_dual(A)}$

In even-dimensional spaces ($EGA2D$, $PGA3DP$) we have to distinguish left and right operations, while in odd-dimensional spaces ($EGA3D$, $PGA2DP$) we don't, since they produce the same result. The "left" operations are typically used on the left-hand side of the binary operation and the "right" operations on the right-hand side of the product operator. The only deviation from that pattern occurs for the definition of the regressive operations, as defined in (18a) or (18b).

In a two-dimensional algebra, like $EGA2D$, the effect of the right dual of a vector u is to turn the vector by +90°, i.e. into the orientation of $\boldsymbol{e}_{12}$, while the left dual turns the vector by −90°, i.e. into the direction against the orientation of $\boldsymbol{e}_{12}$. This statement is valid for both right- and left-handed coordinate systems.

Before we go on to the geometric product, let's return to eqn. (13) for a moment, which provided the definition of a plane as trivector in projective space. We are going to use this example to show what can be done geometrically using the definitions provided above:

The trivector defining a plane can be created in multiple ways, e.g. either by wedge products between three unitized projective points P, Q, R that span the plane as $pl = P \wedge Q \wedge R$ as their common subspace, or alternatively as wedge products between a unitized projective point P and two direction vectors u and v that span the plane as a bivector such that $pl = P \wedge u \wedge v$. In the latter case, the plane pl can be interpreted as bound bivector as suggested by [Browne, 2012]. For this case it is also easy to shift the plane to contain the unitized point $P + x$ instead of P. This can be achieved easily by applying the rules for the wedge product as $pl_{shifted} = P \wedge u \wedge v + x \wedge u \wedge v = (P + x) \wedge u \wedge v$.

The attitude of the plane, i.e. its orientation in space, is given by a bivector that can be calculated using the attitude function $att(pl)$. It is defined for $3D$ projective space as $att(u) = u \vee \overline{\boldsymbol{e}_4}$ and as such can be interpreted as the intersection of the plane with the horizon $H_{3dp} = \overline{\boldsymbol{e}_4}$. It yields the result $att(pl) = t_x \boldsymbol{e}_{23} + t_y \boldsymbol{e}_{31} + t_z \boldsymbol{e}_{12}$.

The definition of a plane as given in eqn. (13) implies that the normal direction n of the plane pl can always simply be recovered from its trivector definition by applying the left weight dual to the plane, such that $n = pl_\star$. The left weight dual operation was introduced in eqn. (33). It can also be found in table 3.

The reverse is also true, i.e. if the plane normal n is known, the trivector representing the plane can be directly found by applying the right bulk dual operation to the normal n, such that $pl_O = n^\star$. The right bulk dual operation was defined in eqn. (30), and can be found in table 3. This operation yields a plane that contains the origin and has the required orientation with n as normal. The signed distance to the origin could now be given as coefficient t_w to the bulk component $\boldsymbol{e}_{312}$ in case the plane needs to be shifted parallel.

As demonstrated, this enables quite a number of relevant geometric operations by applying rather simple operations to the modelled objects in $3D$ PGA.

Now let's move on in more detail to the geometric product. The multiplicative identity of the geometric product and the wedge product is the scalar basis element $\mathbf{1}$:

$$\mathbf{1} \wedge\!\!\!\!\wedge a = a \wedge\!\!\!\!\wedge \mathbf{1} = a \tag{37a}$$

$$\mathbf{1} \wedge a = a \wedge \mathbf{1} = a \tag{37b}$$

The multiplicative identity of the regressive geometric product and the regressive wedge product is the pseudoscalar basis element $\mathbb{1} = I_n$ of the respective algebra:

$$\mathbb{1} \vee\!\!\!\!\vee a = a \vee\!\!\!\!\vee \mathbb{1} = a \tag{38a}$$

$$\mathbb{1} \vee a = a \vee \mathbb{1} = a \tag{38b}$$

The geometric product combines the dimensions that are present in an object representation, e.g. for $PGA2DP$:

$$\begin{aligned} \boldsymbol{e}_1 \wedge\!\!\!\!\wedge \boldsymbol{e}_1 &= 1 \\ \boldsymbol{e}_2 \wedge\!\!\!\!\wedge \boldsymbol{e}_2 &= 1 \\ \boldsymbol{e}_3 \wedge\!\!\!\!\wedge \boldsymbol{e}_3 &= 0 \end{aligned} \tag{39}$$

The regressive geometric product combines the dimensions that are <u>*not*</u> present in an object representation, e.g. for $PGA2DP$:

$$\begin{aligned} \overline{\boldsymbol{e}_1} \vee\!\!\!\!\vee \overline{\boldsymbol{e}_1} &= \mathbb{1} \\ \overline{\boldsymbol{e}_2} \vee\!\!\!\!\vee \overline{\boldsymbol{e}_2} &= \mathbb{1} \\ \overline{\boldsymbol{e}_3} \vee\!\!\!\!\vee \overline{\boldsymbol{e}_3} &= 0 \end{aligned} \tag{40}$$

where eqs. (40) can be shown to be equivalent to eqs. (39) by taking the left complement of both sides and applying the complement definition (18a) to it.

The inner product between multivectors A and B is defined as

$$A \cdot B = A^T \mathbf{G} B \tag{41}$$

with $\mathbf{G}$ as the extended metric and using standard matrix multiplication on the right hand side. It satisfies the identity

$$A \cdot B = \langle A \wedge\!\!\!\wedge \tilde{B} \rangle_0 = \langle B \wedge\!\!\!\wedge \tilde{A} \rangle_0 \tag{42}$$

where $\wedge\!\!\!\wedge$ stands for the geometric product within the grade projection operator $\langle\rangle_k$ for grade k.

Using the various usually used notations in literature the contraction is explicitly defined as

$$A \rfloor B = A \ll B = A_\star \vee B \tag{43a}$$
$$B \lfloor A = B \gg A = B \vee A^\star \tag{43b}$$

with $\star$ denoting the Hodge dual which is formed by the respective complement operation (in spaces of even dimension the left or right complement respectively, and in spaces of odd dimension the complement function regardless of the side of the operand). For blades A and B, they satisfy

$$A \rfloor B = A \ll B = \langle B \wedge\!\!\!\wedge \tilde{A} \rangle_{gr(B)-gr(A)} \tag{44a}$$
$$B \lfloor A = B \gg A = \langle \tilde{A} \wedge\!\!\!\wedge B \rangle_{gr(B)-gr(A)} \tag{44b}$$

and fulfill the requirement that $A \rfloor B = A \lfloor B = A \cdot B$ whenever A and B have the same grade. In this case the contractions reduce to the inner product.

The geometric product between a vector v and a blade B is defined in terms of interior and exterior products, i.e. the right contraction "$\lfloor$", the left contraction "$\rfloor$" and wedge product "$\wedge$" as follows:

$$v \wedge\!\!\!\wedge B = B \lfloor v + v \wedge B \tag{45a}$$
$$B \wedge\!\!\!\wedge v = v \rfloor B + B \wedge v \tag{45b}$$

or sometimes expressed using the right shift "$\gg$" or left shift "$\ll$" operators for the contractions (as implemented in the ga library)

$$v \wedge\!\!\!\wedge B = B \gg v + v \wedge B \tag{46a}$$
$$B \wedge\!\!\!\wedge v = v \ll B + B \wedge v \tag{46b}$$

or sometimes by exclusively using the operations of the exterior algebra (wedge product "$\wedge$", bulk dual "$\star$" and regressive wedge product "$\vee$")

$$v \wedge\!\!\!\wedge B = B \vee v^{\star} + v \wedge B \tag{47a}$$

$$B \wedge\!\!\!\wedge v = v_{\star} \vee B + B \wedge v \tag{47b}$$

as an alternative. Each one is equivalent.

Be carefull, when using the equivalent formulas like the right-hand side of equations (47) when deriving regressive products. Resulting formulas may not be valid! The equivalent formulas for the regressive products must be derived directly from the regressive product table. To give one example, here is the corresponding formula for the regressive geometric product for the case of a regressive product between a vector v and a bivector B:

$$v \vee\!\!\!\vee B = -B_{\star} \wedge v + \mathrm{rcmt}(v, B) \tag{48a}$$

$$B \vee\!\!\!\vee v = -v \wedge B^{\star} + \mathrm{rcmt}(B, v) \tag{48b}$$

It is obviously not the result one would get from creating the regressive versions directly from the right-hand side of equations (47). The reason is that the specific pre-conditions for the validity of the equivalence between the geometric product and the corresponding right-hand sides may change when taking the complements.

Every vector v can be decomposed into parts $v_{\parallel}$ parallel to a k-blade B and $v_{\perp}$ perpendicular to B, such that $v = v_{\parallel} + v_{\perp}$. For equation (46a): $B \gg v$ is a $(k-1)$-blade. If $v_{\parallel} \neq 0$ then $B \gg v$ represents a subspace of B. $v \wedge B$ is a $(k+1)$-blade. If $v_{\perp} \neq 0$ then $v \wedge B$ represents $span(v, B)$.
For arguments of equal grade the contractions reduce to the dot product, so that one can write specifically for two vectors a and b:

$$a \wedge\!\!\!\wedge b = a \cdot b + a \wedge b \tag{49}$$

[Lengyel, 2025] has shown that the geometric product can be derived by using operations from Grassmann algebra exclusively. These are the wedge product, the complement and the duality operations. This demonstrates that the geometric product is not a fundamental operation in its own right, but can be derived from more fundamental basic operations.

For that purpose Lengyel introduced the transwedge product of order k by following definition:

$$a \underset{k}{\wedge\!\!\!\wedge} b = \sum_{c \in \mathcal{B}_k} (\underline{c} \vee a) \wedge (b \vee c^\star) \tag{50}$$

where $\mathcal{B}_k$ is the set of all basis elements of order k. For $EGA3D$ it means for example $\mathcal{B}_0 = \{1\}$, $\mathcal{B}_1 = \{\boldsymbol{e}_1, \boldsymbol{e}_2, \boldsymbol{e}_3\}$, $\mathcal{B}_2 = \{\boldsymbol{e}_{23}, \boldsymbol{e}_{31}, \boldsymbol{e}_{12}\}$, and $\mathcal{B}_3 = \{\boldsymbol{e}_{123}\}$. From these expressions the geometric product can be formed by

$$a \wedge b = \sum_{k=0}^{n} (-1)^{k(k-1)/2} a \underset{k}{\wedge\!\!\!\wedge} b \tag{51}$$

For $k = 0$ this yields the outer product (wedge product). For higher values of k other products are generated which sum up to the full geometric product. Thus the transwedge products helps to decompose the geometric product. The most interesting for applications are the wedge product resulting for $k = 0$ and the transwedge product of order $k = 1$.

The regressive wedge product in equation (50) can be replaced by the wedge product using the complement-based definitions given in equations (16) to (20b). That means for example that equation (50) can be transformed to the equivalent expression

$$a \underset{k}{\wedge\!\!\!\wedge} b = \sum_{c \in \mathcal{B}_k} (\underline{c \wedge \overline{a}}) \wedge (\underline{\overline{b} \wedge \overline{c^\star}}) \tag{52}$$

expressing it by using the wedge product exclusively.

Since every product can by assigned a corresponding regressive product in analogy to definition (18a) or (18b), the transwedge product (50) also has a regressive form:

$$a \underset{k}{\vee\!\!\!\vee} b \equiv \overline{\underline{a} \underset{k}{\wedge\!\!\!\wedge} \underline{b}} = \sum_{c \in \mathcal{B}_k} \overline{(\underline{c} \vee \underline{a}) \wedge (\underline{b} \vee c^\star)} \tag{53}$$

which in turn can be expressed in terms of the wedge product exclusively as:

$$a \underset{k}{\vee\!\!\!\vee} b = \sum_{c \in \mathcal{B}_k} \overline{(\underline{c \wedge a}) \wedge (\underline{b \wedge \overline{c^\star}})} \tag{54}$$

Equations (52) and (54) are used for calculation of the product tables of the (regressive) transwedge products of order k and the resulting (regressive) geometric products in `ga/ga_prdxpr`. The transwedge product is very useful to show that the geometric product can be derived from primitives of Grassmann algebra. When (regressive) transwedge product expressions are calculated, it can be shown that each transwedge product and the resulting coefficient expressions are equivalent to specific combinations of wedge-, commutator-, dot-products, and their regressive counterparts. This was done in `ga/src_prdxpr/ga_pga2dp_ops_products.hpp` for the (regressive) transwedge product of order $k = 1$ and as well in `ga/src_prdxpr/ga_pga3dp_ops_products.hpp` to show the equivalence.

$\boldsymbol{u}$	$\mathbf{1}$	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$\boldsymbol{e}_3$	$\boldsymbol{e}_4$	$\boldsymbol{e}_{41}$	$\boldsymbol{e}_{42}$	$\boldsymbol{e}_{43}$	$\boldsymbol{e}_{23}$	$\boldsymbol{e}_{31}$	$\boldsymbol{e}_{12}$	$\boldsymbol{e}_{423}$	$\boldsymbol{e}_{431}$	$\boldsymbol{e}_{412}$	$\boldsymbol{e}_{321}$	$\mathbb{1}$
$\boldsymbol{u}\cdot\boldsymbol{u}$	$\mathbf{1}$	$\mathbf{1}$	$\mathbf{1}$	$\mathbf{1}$	0	0	0	0	$\mathbf{1}$	$\mathbf{1}$	$\mathbf{1}$	0	0	0	$\mathbf{1}$	0
$\boldsymbol{u}\circ\boldsymbol{u}$	0	0	0	0	$\mathbb{1}$	$\mathbb{1}$	$\mathbb{1}$	$\mathbb{1}$	0	0	0	$\mathbb{1}$	$\mathbb{1}$	$\mathbb{1}$	0	$\mathbb{1}$

Table 1: Dot and regressive dot products in $PGA3DP$ for every basis element.

$\boldsymbol{u}$	$\mathbf{1}$	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$\boldsymbol{e}_3$	$\boldsymbol{e}_4$	$\boldsymbol{e}_{41}$	$\boldsymbol{e}_{42}$	$\boldsymbol{e}_{43}$	$\boldsymbol{e}_{23}$	$\boldsymbol{e}_{31}$	$\boldsymbol{e}_{12}$	$\boldsymbol{e}_{423}$	$\boldsymbol{e}_{431}$	$\boldsymbol{e}_{412}$	$\boldsymbol{e}_{321}$	$\mathbb{1}$
$\overline{\boldsymbol{u}}$	$\mathbb{1}$	$\boldsymbol{e}_{423}$	$\boldsymbol{e}_{431}$	$\boldsymbol{e}_{412}$	$\boldsymbol{e}_{321}$	$-\boldsymbol{e}_{23}$	$-\boldsymbol{e}_{31}$	$-\boldsymbol{e}_{12}$	$-\boldsymbol{e}_{41}$	$-\boldsymbol{e}_{42}$	$-\boldsymbol{e}_{43}$	$-\boldsymbol{e}_1$	$-\boldsymbol{e}_2$	$-\boldsymbol{e}_3$	$-\boldsymbol{e}_4$	$\mathbf{1}$
$\underline{\boldsymbol{u}}$	$\mathbb{1}$	$-\boldsymbol{e}_{423}$	$-\boldsymbol{e}_{431}$	$-\boldsymbol{e}_{412}$	$-\boldsymbol{e}_{321}$	$-\boldsymbol{e}_{23}$	$-\boldsymbol{e}_{31}$	$-\boldsymbol{e}_{12}$	$-\boldsymbol{e}_{41}$	$-\boldsymbol{e}_{42}$	$-\boldsymbol{e}_{43}$	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$\boldsymbol{e}_3$	$\boldsymbol{e}_4$	$\mathbf{1}$

Table 2: Right and left complements in $PGA3DP$ for every basis element.

$\boldsymbol{u}$	$\mathbf{1}$	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$\boldsymbol{e}_3$	$\boldsymbol{e}_4$	$\boldsymbol{e}_{41}$	$\boldsymbol{e}_{42}$	$\boldsymbol{e}_{43}$	$\boldsymbol{e}_{23}$	$\boldsymbol{e}_{31}$	$\boldsymbol{e}_{12}$	$\boldsymbol{e}_{423}$	$\boldsymbol{e}_{431}$	$\boldsymbol{e}_{412}$	$\boldsymbol{e}_{321}$	$\mathbb{1}$
$\boldsymbol{u}^\star$	$\mathbb{1}$	$\boldsymbol{e}_{423}$	$\boldsymbol{e}_{431}$	$\boldsymbol{e}_{412}$	0	0	0	0	$-\boldsymbol{e}_{41}$	$-\boldsymbol{e}_{42}$	$-\boldsymbol{e}_{43}$	0	0	0	$-\boldsymbol{e}_4$	0
$\boldsymbol{u}_\star$	$\mathbb{1}$	$-\boldsymbol{e}_{423}$	$-\boldsymbol{e}_{431}$	$-\boldsymbol{e}_{412}$	0	0	0	0	$-\boldsymbol{e}_{41}$	$-\boldsymbol{e}_{42}$	$-\boldsymbol{e}_{43}$	0	0	0	$\boldsymbol{e}_4$	0
$\boldsymbol{u}^{\star\star}$	0	0	0	0	$\boldsymbol{e}_{321}$	$-\boldsymbol{e}_{23}$	$-\boldsymbol{e}_{31}$	$-\boldsymbol{e}_{12}$	0	0	0	$-\boldsymbol{e}_1$	$-\boldsymbol{e}_2$	$-\boldsymbol{e}_3$	0	$\mathbf{1}$
$\boldsymbol{u}_{\star\star}$	0	0	0	0	$-\boldsymbol{e}_{321}$	$-\boldsymbol{e}_{23}$	$-\boldsymbol{e}_{31}$	$-\boldsymbol{e}_{12}$	0	0	0	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$\boldsymbol{e}_3$	0	$\mathbf{1}$

Table 3: Right/left bulk duals and right/left weight duals in $PGA3DP$ for every basis element (=complement applied to bulk and weight).

$\boldsymbol{u}$	$\mathbf{1}$	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$\boldsymbol{e}_3$	$\boldsymbol{e}_4$	$\boldsymbol{e}_{41}$	$\boldsymbol{e}_{42}$	$\boldsymbol{e}_{43}$	$\boldsymbol{e}_{23}$	$\boldsymbol{e}_{31}$	$\boldsymbol{e}_{12}$	$\boldsymbol{e}_{423}$	$\boldsymbol{e}_{431}$	$\boldsymbol{e}_{412}$	$\boldsymbol{e}_{321}$	$\mathbb{1}$
$\widetilde{\boldsymbol{u}}$	$\mathbf{1}$	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$\boldsymbol{e}_3$	$\boldsymbol{e}_4$	$-\boldsymbol{e}_{41}$	$-\boldsymbol{e}_{42}$	$-\boldsymbol{e}_{43}$	$-\boldsymbol{e}_{23}$	$-\boldsymbol{e}_{31}$	$-\boldsymbol{e}_{12}$	$-\boldsymbol{e}_{423}$	$-\boldsymbol{e}_{431}$	$-\boldsymbol{e}_{412}$	$-\boldsymbol{e}_{321}$	$\mathbb{1}$
$\underset{\sim}{\boldsymbol{u}}$	$\mathbf{1}$	$-\boldsymbol{e}_1$	$-\boldsymbol{e}_2$	$-\boldsymbol{e}_3$	$-\boldsymbol{e}_4$	$-\boldsymbol{e}_{41}$	$-\boldsymbol{e}_{42}$	$-\boldsymbol{e}_{43}$	$-\boldsymbol{e}_{23}$	$-\boldsymbol{e}_{31}$	$-\boldsymbol{e}_{12}$	$\boldsymbol{e}_{423}$	$\boldsymbol{e}_{431}$	$\boldsymbol{e}_{412}$	$\boldsymbol{e}_{321}$	$\mathbb{1}$

Table 4: Reverse and regressive reverse in $PGA3DP$ for every basis element.

$\boldsymbol{u}$	$\mathbf{1}$	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$\boldsymbol{e}_3$	$\boldsymbol{e}_{31}$	$\boldsymbol{e}_{32}$	$\boldsymbol{e}_{12}$	$\mathbb{1}$
$\boldsymbol{u} \cdot \boldsymbol{u}$	$\mathbf{1}$	$\mathbf{1}$	$\mathbf{1}$	0	0	0	$\mathbf{1}$	0
$\boldsymbol{u} \circ \boldsymbol{u}$	0	0	0	$\mathbb{1}$	$\mathbb{1}$	$\mathbb{1}$	0	$\mathbb{1}$

Table 5: Dot and regressive dot products in $PGA2DP$ for every basis element.

$\boldsymbol{u}$	$\mathbf{1}$	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$\boldsymbol{e}_3$	$\boldsymbol{e}_{31}$	$\boldsymbol{e}_{32}$	$\boldsymbol{e}_{12}$	$\mathbb{1}$
$\overline{\boldsymbol{u}}$	$\mathbb{1}$	$\boldsymbol{e}_{32}$	$-\boldsymbol{e}_{31}$	$-\boldsymbol{e}_{12}$	$-\boldsymbol{e}_2$	$\boldsymbol{e}_1$	$-\boldsymbol{e}_3$	$\mathbf{1}$

Table 6: Complements in $PGA2DP$ for every basis element.

$\boldsymbol{u}$	$\mathbf{1}$	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$\boldsymbol{e}_3$	$\boldsymbol{e}_{31}$	$\boldsymbol{e}_{32}$	$\boldsymbol{e}_{12}$	$\mathbb{1}$
$\boldsymbol{u}^\star$	$\mathbb{1}$	$\boldsymbol{e}_{32}$	$-\boldsymbol{e}_{31}$	0	0	0	$-\boldsymbol{e}_3$	0
$\boldsymbol{u}^\star$	0	0	0	$-\boldsymbol{e}_{12}$	$-\boldsymbol{e}_2$	$\boldsymbol{e}_1$	0	$\mathbf{1}$

Table 7: Bulk duals and weight duals in $PGA2DP$ for every basis element (=complement applied to bulk and weight).

$\boldsymbol{u}$	$\mathbf{1}$	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$\boldsymbol{e}_3$	$\boldsymbol{e}_{31}$	$\boldsymbol{e}_{32}$	$\boldsymbol{e}_{12}$	$\mathbb{1}$
$\widetilde{\boldsymbol{u}}$	$\mathbf{1}$	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$\boldsymbol{e}_3$	$-\boldsymbol{e}_{31}$	$-\boldsymbol{e}_{32}$	$-\boldsymbol{e}_{12}$	$-\mathbb{1}$
$\underset{\sim}{\boldsymbol{u}}$	$-\mathbf{1}$	$-\boldsymbol{e}_1$	$-\boldsymbol{e}_2$	$-\boldsymbol{e}_3$	$\boldsymbol{e}_{31}$	$\boldsymbol{e}_{32}$	$\boldsymbol{e}_{12}$	$\mathbb{1}$

Table 8: Reverse and regressive reverse in $PGA2DP$ for every basis element.

1.3 Modelling motion

For translational motion in physical contexts we typically use the following naming convention for scalars or vectors:

$$\begin{aligned} s(t) &= s \\ \dot{s}(t) &= v(t) = \dot{s} = v \\ \ddot{s}(t) &= \dot{v}(t) = a(t) = \ddot{s} = \dot{v} = a \end{aligned}$$

while for rotational motion we typically use:

$$\begin{aligned} \phi(t) &= \phi \\ \dot{\phi}(t) &= \omega(t) = \dot{\phi} = \omega \\ \ddot{\phi}(t) &= \dot{\omega}(t) = \ddot{\phi} = \dot{\omega} \end{aligned}$$

Especially in PGA, we will be able to combine the translational and rotational aspects in one physical variable. Thus we will use a convention of using capital letters for physical quantities to distinguish them from the classical convention, especially so for bivectors and even-grade multivectors that do not occur in classical descriptions. We will use in PGA:

$B(t) = B$	(bivector)
$\dot{B}(t) = \Omega(t) = \dot{B} = \Omega$	(rate of change of bivector, "bivector speed")
$\ddot{B}(t) = \dot{\Omega}(t) = \ddot{B} = \dot{\Omega}$	(rate of change of bivector speed)

In order to model motion in PGA we need to work with the exponential function and apply it in the context of the geometric product (symbol "$\wedge$") and the regressive geometric product (symbol "$\vee$").

$$\exp(x) = \sum_{n=0}^{\infty} \frac{x^n}{n!} = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \frac{x^5}{5!} + \frac{x^6}{6!} + \frac{x^7}{7!} + \cdots \tag{55}$$

$$\cos(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n}}{(2n)!} = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \frac{x^8}{8!} - \frac{x^{10}}{10!} + \frac{x^{12}}{12!} - \frac{x^{14}}{14!} + \cdots \tag{56}$$

$$\sin(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n+1}}{(2n+1)!} = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \frac{x^9}{9!} - \frac{x^{11}}{11!} + \frac{x^{13}}{13!} - \frac{x^{15}}{15!} + \cdots \tag{57}$$

If we substitute $x = i\phi$ with i as the imaginary unit that squares to -1, we get Euler's formula:

$$\begin{aligned}
\exp(i\phi) &= \sum_{n=0}^{\infty} \frac{(i\phi)^n}{n!} \\
&= 1 + i\phi + \frac{(i\phi)^2}{2!} + \frac{(i\phi)^3}{3!} + \frac{(i\phi)^4}{4!} + \frac{(i\phi)^5}{5!} + \frac{(i\phi)^6}{6!} + \frac{(i\phi)^7}{7!} + \cdots && (58) \\
&= 1 + i\phi - \frac{\phi^2}{2!} - i\frac{\phi^3}{3!} + \frac{\phi^4}{4!} + i\frac{\phi^5}{5!} - \frac{\phi^6}{6!} - i\frac{\phi^7}{7!} + \cdots \\
&= 1 - \frac{\phi^2}{2!} + \frac{\phi^4}{4!} - \frac{\phi^6}{6!} + \cdots + i(\phi - \frac{\phi^3}{3!} + \frac{\phi^5}{5!} - \frac{\phi^7}{7!} + \cdots) && (59)
\end{aligned}$$

$$\exp(i\phi) = \cos(\phi) + i\sin(\phi) \tag{60}$$

Applying this for geometric algebra we have to distinguish between calculating the powers in the Taylor series using the geometric product (in EGA) and using the regressive geometric product (in PGA). For the rotation case we assume a unitized bivector B and a scalar ϕ. Based on the geometric product we know that $B \wedge B = -\mathbf{1}$, and for its regressive counterpart we know that $B \vee B = -\mathbb{1}$. Thus, B plays a role equivalent to i in both cases. In addition, taking any value to the power of 0 results in the neutral element of the product, which is $\mathbf{1}$ for the geometric product, and $\mathbb{1}$ for the regressive geometric product. Taking the powers then works like this:

$$\begin{aligned}
&\cdots \\
\frac{(B\phi)^2_{\wedge}}{2!} &= \frac{1}{2!}(B\phi \wedge B\phi) = \frac{\phi^2}{2!}(B \wedge B) = \frac{\phi^2}{2!}(-\mathbf{1}) = -\frac{\phi^2}{2!} && (61a) \\
\frac{(B\phi)^3_{\wedge}}{3!} &= \frac{\phi^3}{3!}(B \wedge B \wedge B) = \frac{\phi^3}{3!}(-\mathbf{1} \wedge B) = -\frac{\phi^3}{3!}B && (61b) \\
\frac{(B\phi)^4_{\wedge}}{4!} &= \frac{\phi^4}{4!}(B \wedge B \wedge B \wedge B) = \frac{\phi^4}{4!}(-\mathbf{1} \wedge -\mathbf{1}) = \frac{\phi^4}{4!} && (61c) \\
&\cdots \\
\frac{(B\phi)^2_{\vee}}{2!} &= \frac{1}{2!}(B\phi \vee B\phi) = \frac{\phi^2}{2!}(B \vee B) = \frac{\phi^2}{2!}(-\mathbb{1}) = -\frac{\phi^2}{2!}\mathbb{1} && (61d) \\
\frac{(B\phi)^3_{\vee}}{3!} &= \frac{\phi^3}{3!}B \vee B \vee B = \frac{\phi^3}{3!}(-\mathbb{1} \vee B) = -\frac{\phi^3}{3!}B && (61e) \\
\frac{(B\phi)^4_{\vee}}{4!} &= \frac{\phi^4}{4!}B \vee B \vee B \vee B = \frac{\phi^4}{4!}(-\mathbb{1} \vee -\mathbb{1}) = \frac{\phi^4}{4!}\mathbb{1} && (61f) \\
&\cdots
\end{aligned}$$

Thus, the result of exponentiating a bivector B is:

$$\begin{aligned}
\exp_{\wedge}(B\phi) &= \sum_{n=0}^{\infty} \frac{(B\phi)^n_{\wedge}}{n!} \\
&= 1 + B\phi + \frac{(B\phi)^2_{\wedge}}{2!} + \frac{(B\phi)^3_{\wedge}}{3!} + \frac{(B\phi)^4_{\wedge}}{4!} + \frac{(B\phi)^5_{\wedge}}{5!} + \frac{(B\phi)^6_{\wedge}}{6!} + \cdots \\
&= 1 + B\phi - \frac{\phi^2}{2!} - B\frac{\phi^3}{3!} + \frac{\phi^4}{4!} + B\frac{\phi^5}{5!} - \frac{\phi^6}{6!} - B\frac{\phi^7}{7!} + \cdots \\
&= 1 - \frac{\phi^2}{2!} + \frac{\phi^4}{4!} - \frac{\phi^6}{6!} + \cdots + B(\phi - \frac{\phi^3}{3!} + \frac{\phi^5}{5!} - \frac{\phi^7}{7!} + \cdots)
\end{aligned} \tag{62}$$

$$\exp_{\wedge}(B\phi) = \cos(\phi) + \sin(\phi)B \tag{63}$$

The last step of moving B after the brackets is allowed, since scalar factors commute with any object using the regular product. The result structurally is an even-grade multivector with a scalar and a bivector component. We get the correct sign and angle of rotation if we replace ϕ by $-\frac{1}{2}\phi$, which was not considered above to keep the formulas as simple as possible. The substitution yields the final form of a rotor R in EGA with B as unitized bivector and ϕ the rotation angle:

$$\begin{aligned}
R = \exp_{\wedge}(-\frac{\phi}{2}B) &= \cos(-\frac{\phi}{2}) + \sin(-\frac{\phi}{2})B \\
R &= \cos(\frac{\phi}{2}) - \sin(\frac{\phi}{2})B
\end{aligned} \tag{64}$$

For the case using the regressive geometric product it becomes:

$$\begin{aligned}
\exp_{\vee}(B\phi) &= \sum_{n=0}^{\infty} \frac{(B\phi)^n_{\vee}}{n!} \\
&= \mathbb{1} + B\phi + \frac{(B\phi)^2_{\vee}}{2!} + \frac{(B\phi)^3_{\vee}}{3!} + \frac{(B\phi)^4_{\vee}}{4!} + \frac{(B\phi)^5_{\vee}}{5!} + \frac{(B\phi)^6_{\vee}}{6!} + \cdots \\
&= \mathbb{1} + B\phi - \mathbb{1}\frac{\phi^2}{2!} - B\frac{\phi^3}{3!} + \mathbb{1}\frac{\phi^4}{4!} + B\frac{\phi^5}{5!} - \mathbb{1}\frac{\phi^6}{6!} - B\frac{\phi^7}{7!} + \cdots \\
&= \mathbb{1}(1 - \frac{\phi^2}{2!} + \frac{\phi^4}{4!} - \frac{\phi^6}{6!} + \cdots) + B(\phi - \frac{\phi^3}{3!} + \frac{\phi^5}{5!} - \frac{\phi^7}{7!} + \cdots)
\end{aligned} \tag{65}$$

$$\exp_{\vee}(B\phi) = \cos(\phi)\mathbb{1} + \sin(\phi)B \tag{66}$$

The last step of moving $\mathbb{1}$ and B after the brackets is allowed, since scalar factors, like the results of the trigonometric functions, commute with any object using the regular product. The resulting motor structurally is an even-grade multivector with a bivector and a pseudoscalar of grade 4 for $3D$ PGA.

In $2D$ PGA we have the same expression, but the argument of the exponential to create a motor for rotation is not a bivector B, but a unitized point P. Here $P \vee P = -\mathbb{1}$ is valid, and equation (66) becomes $\exp_{\vee}(P\phi) = \cos(\phi)\mathbb{1} + \sin(\phi)P$ instead, which leads to a motor M that structurally is an odd-grade multivector with a vector and a pseudoscalar of grade 3. The resulting rotation direction of the motor depends on the chosen pseudoscalar. The pseudoscalar for $PGA2DP$ was chosen as $\boldsymbol{e}_{321}$ in order to make signs of rotation angles (and thus turning directions) consistent between $2D$ and $3D$ PGA.

To model a translation $\delta = (\delta_x, \delta_y, \delta_z, 0)$ in $3D$ PGA we use a special bivector $B = B_{tra}$ that does not square to $-\mathbb{1}$, but instead squares to zero. That makes sure that the expansion of the Taylor series of the exponential function $exp_{\vee}(B_{tra})$ only contains the first two terms. The bivector is resulting from two consequtive reflections across parallel planes perpendicular to δ with the distance $\frac{1}{2}\|\delta\|_{\bullet}$ relative to each other. They intersect in a line at infinity that is used as "rotation axis". This however yields a translation, since the axis is infinitely far away. The derivation of the formula for the bivector will be done later in this chapter (see eqn. (106)). We use $B_{tra} = \delta^{\star} \vee \overline{\boldsymbol{e}_4} = \delta^{\star} \vee H_{3dp} = att(\delta^{\star})$. This yields $B_{tra} = (\delta_x \boldsymbol{e}_{23} + \delta_y \boldsymbol{e}_{31} + \delta_z \boldsymbol{e}_{12})$. The resulting motor is $M = exp_{\vee}(B_{tra}) = \mathbb{1} + \frac{1}{2}B_{tra} = \mathbb{1} + \frac{1}{2}(0, 0, 0, \delta_x, \delta_y, \delta_z)$. The motor M is an even-grade multivector containing a pseudoscalar component, which is a quadvector in $3D$ PGA, and a bivector component.

To model a translation $\delta = (\delta_x, \delta_y, 0)$ in $2D$ PGA we need a special vector as argument of the exponential. It represents a direction to a point at infinity. The "rotation" around that infinitely far away point generates a translation (actually it is resulting from two consequtive reflections across parallel lines perpendicular to δ with the distance $\frac{1}{2}\|\delta\|_{\bullet}$ relative to each other). The deriviation of the special vector will be done later in this section (see eqn. (113)). The argument of the exponential $exp_{\vee}$ for a translation is the direction vector $P = \delta^{\star} \vee \overline{\boldsymbol{e}_3} = \delta^{\star} \vee H_{2dp} = att(\delta^{\star}) = (-\delta_y \boldsymbol{e}_1 + \delta_x \boldsymbol{e}_2)$. P points in a direction perpendicular to the translation δ towards infinity. The main difference of this special point P compared to the one used for the rotation above is that $P_z = 0$. This results in $P \vee P = 0$ which causes the expansion of the exponential to only contain the first two terms: $M = exp_{\vee}(P) = \mathbb{1} + \frac{1}{2}P = \mathbb{1} + \frac{1}{2}(-\delta_y, \delta_x, 0)$. The motor M is an odd-grade multivector containing a pseudoscalar, which is a trivector in $2D$ PGA, and a vector component, i.e. it is structurally the same as for the rotation case in $2D$ PGA above.

Eqn. (63) allows us to model rotations in EGA, where the bivector B is the complement of the axis of rotation and $\theta = -2\phi$ is the angle of the rotation

around that axis. If we directly want to specify the angle of rotation θ we need to substitute ϕ by $-\frac{\phi}{2}$ in eqn. (63). The rotated object X can then be obtained from the original object X_0 by:

$$X = R \wedge X_0 \wedge \widetilde{R} = \exp_{\wedge}(-\frac{\phi}{2}B) \wedge X_0 \wedge \exp_{\wedge}(\frac{\phi}{2}B) \quad (\text{for } EGA) \tag{67}$$

If we want to model combined translations and rotations in PGA, we need to look into another extension of the exponential function. We follow the approach defined by [Lengyel, 2024b] in chapter 3.2 and 3.6.2, where it is shown that a general motor in PGA can be expressed as a screw motion, i.e. a rotation around a unitized line l by an angle ϕ and a translation along l by a displacement δ as

$$\begin{aligned} M = \exp_{\vee}\left[(\frac{\delta}{2}\mathbf{1} + \frac{\phi}{2}\mathbb{1}) \vee l\right] &= \cos_{\vee}(\frac{\delta}{2}\mathbf{1} + \frac{\phi}{2}\mathbb{1}) + \sin_{\vee}(\frac{\delta}{2}\mathbf{1} + \frac{\phi}{2}\mathbb{1}) \vee \boldsymbol{l} \\ &= -\frac{\delta}{2}\mathbf{1}\sin(\frac{\phi}{2}) + \mathbb{1}\cos(\frac{\phi}{2}) + \left(\frac{\delta}{2}\mathbf{1}\cos(\frac{\phi}{2}) + \mathbb{1}\sin(\frac{\phi}{2})\right) \vee \boldsymbol{l} \\ M &= -\frac{\delta}{2}\mathbf{1}\sin(\frac{\phi}{2}) - \boldsymbol{l}^{\star}\frac{\delta}{2}\cos(\frac{\phi}{2}) + \boldsymbol{l}\sin(\frac{\phi}{2}) + \mathbb{1}\cos(\frac{\phi}{2}) \end{aligned} \tag{68}$$

where M structurally is an even grade multivector with scalar, bivector and pseudoscalar parts in $3D$ PGA. The argument in the exponential function again results in a bivector. This can be derived from inspecting the product table for the regressive geometric product. However, this time we only have a common factor of $\frac{1}{2}$, and not $-\frac{1}{2}$ as in EGA. This will impact a sign for the transformation in the regressive case for PGA compared to the corresponding transformations in EGA. The resulting transformation is then obtained from

$$X = M \vee X_0 \vee \underset{\sim}{M} \quad (\text{for } PGA) \tag{69}$$

We can think of the screw motion in $3D$ PGA as a combination of a rotation by an angle ϕ around the unitized line l and by a translation from a displacement δ along that same line l. The latter part results from a rotation around a line at infinity given by $l^{\star}$. Thus the bivector representing the line l can always be split into the parts creating the rotation (movement around that line) and the parts creating the translation (movement along that line).

In order to dive deeper into modelling of motions we first start with the EGA case. It is identical to the derivation shown in [Dorst and De Keninck, 2023] for their plane-based approach. This will help to better recognize the differences to the PGA approach used here.

Before going into more detail, we need to prepare some formulas we later need for our derivations:

For EGA the rotor will now be written as M instead of R to make it easy to compare to our PGA-derivations using motors M later on. M as a rotor in EGA is an orthonomal transformation and thus satisfies

$$M \wedge \widetilde{M} = \widetilde{M} \wedge M = 1 \tag{70}$$

(remember: $M^{-1} = \widetilde{M}$), and therefore

$$\frac{d}{dt}(M \wedge \widetilde{M}) = \dot{M} \wedge \widetilde{M} + M \wedge \dot{\widetilde{M}} = 0 \tag{71}$$

It follows that $M \wedge \dot{\widetilde{M}} = -\dot{M} \wedge \widetilde{M}$, which results in

$$\dot{\widetilde{M}} = -\widetilde{M} \wedge \dot{M} \wedge \widetilde{M} \tag{72}$$

Starting from $\widetilde{M} \wedge M = 1$ instead yields the same result for $\dot{\widetilde{M}}$.

In order to model rigid body mechanics we consider two coordinate frames:

- A *world frame* (index w) that is normally chosen to be an inertial frame.
- A *body frame* (index b) that is attached to a rigid body. Typically at its center of gravity, but this is not strictly required. Rigid body's internal forces are in equilibrium and thus need not be considered explicitly. It is sufficient to describe the translational motion of the body's center of gravity and its rotational motion with respect to the center of gravity. The resulting motion of the body frame relative to the world frame then fully covers rigid body motion (kinematics and dynamics), which depends on distribution of the mass of the body as well as on external forces and moments acting on the body.
- To describe the motion we use a time dependend motor $M = M(t)$. At time $t = 0$ we call it $M_0 = M(t = 0)$. M_0 brings an element X of the body to its position X_0 at time $t = 0$.

World frame: From eqn. (64) we know the form of a general rotor in EGA and use the ansatz for a time dependent rotor $M(t)$ after the the object X was brought into its initial position by M_0 as $X_0 = M \wedge X \wedge \widetilde{M}$. If there is no difference between world and body frames at $t = 0$ then $M_0 = 1$:

$$M(t) = exp_{\wedge}(-\frac{1}{2}B_w(t)) \wedge M_0 \tag{73}$$

with $B_w(t)$ as a time-dependent bivector that models the rotational plane and the turning angle. The derivative of M with respect to time t is

$$\dot{M}(t) = -\frac{1}{2}\dot{B}_w(t) \wedge exp_{\wedge}(-\frac{1}{2}B_w(t)) \wedge M_0 = -\frac{1}{2}\dot{B}_w(t) \wedge M(t) \tag{74}$$

where we use that the exponential of the bivector and the bivector rate commute *(tbd: show why this is the case)*. Using $\Omega_w = \dot{B}_w(t)$ as the bivector rate of change in the *world frame*, we get

$$\dot{M} = -\frac{1}{2}\Omega_w \wedge M \tag{75}$$

By multiplying this from the right with "$\wedge \widetilde{M}$" this can transformed to

$$\Omega_w = -2\dot{M} \wedge \widetilde{M} \tag{76}$$

Body frame: Here we first move the object in the *body frame* and then transform it into the in the world frame by the transformation M_0. In this case eqn. (73) becomes

$$M(t) = M_0 \wedge exp_{\wedge}(-\frac{1}{2}B_b(t)) \tag{77}$$

which in turn restults in

$$\dot{M}(t) = -\frac{1}{2}M_0 \wedge exp_{\wedge}(-\frac{1}{2}B_b(t)) \wedge \dot{B}_b(t) = -\frac{1}{2}M(t) \wedge \dot{B}_b(t) \tag{78}$$

while eqn. (75) for the bivector rate of change in the *body frame* becomes

$$\dot{M} = -\frac{1}{2}M \wedge \Omega_b \tag{79}$$

using $\Omega_b = \dot{B}_b(t)$ as the bivector rate of change in the *body frame*. This can be transformed by multiplying it from the left by "$\widetilde{M} \wedge$" to obtain

$$\Omega_b = -2\widetilde{M} \wedge \dot{M} \tag{80}$$

From equations (75) and (79) we can see how world and body frame rates are connected to each other: $\Omega_w \wedge M = M \wedge \Omega_b$, which can be transformed into

$$\Omega_w = M \wedge \Omega_b \wedge \widetilde{M} \tag{81a}$$

$$\Omega_b = \widetilde{M} \wedge \Omega_w \wedge M \tag{81b}$$

Now we derive the rate of change of a point $X(t)$, i.e. its speed $\dot{X}(t)$, in the *world frame*, first assuming that $X(t=0) = X_0 = const.$, i.e. as a constant element that is transformed to become X by a time dependent motor $M(t)$. During the derivation we will use the associativity of the geometric product and the neutral element 1 of the geometric product. We can write

$$\begin{aligned}
\dot{X}(t) &= \frac{d}{dt}X(t) = \frac{d}{dt}(M \wedge X_0 \wedge \widetilde{M}) \\
&= \dot{M} \wedge X_0 \wedge \widetilde{M} + M \wedge X_0 \wedge \dot{\widetilde{M}} \\
&\overset{(72)}{=} \dot{M} \wedge 1 \wedge X_0 \wedge \widetilde{M} - M \wedge X_0 \wedge (\widetilde{M} \wedge \dot{M} \wedge \widetilde{M}) \\
&\overset{(70)}{=} \dot{M} \wedge (\widetilde{M} \wedge M) \wedge X_0 \wedge \widetilde{M} - (M \wedge X_0 \wedge \widetilde{M}) \wedge \dot{M} \wedge \widetilde{M} \\
&= (\dot{M} \wedge \widetilde{M}) \wedge (M \wedge X_0 \wedge \widetilde{M}) - (M \wedge X_0 \wedge \widetilde{M}) \wedge (\dot{M} \wedge \widetilde{M})
\end{aligned}$$

From eqn. (76) we can conclude that $-\frac{1}{2}\Omega_w(t) = \dot{M} \wedge \widetilde{M}$, resulting in

$$\begin{aligned}
\dot{X}(t) &= (-\frac{1}{2}\Omega_w(t)) \wedge X(t) - X(t) \wedge (-\frac{1}{2}\Omega_w(t)) \\
&= -\frac{1}{2}(\Omega_w \wedge X - X \wedge \Omega_w) \\
&= -cmt(\Omega_w, X) \\
\dot{X}_w &= cmt(X_w, \Omega_w) \qquad (82)
\end{aligned}$$

with $cmt(X_w, \Omega_w)$ as the commutator product between X_w and Ω_w.

So, if the bivector rate of change Ω_w is known, the rate of change of X_w, which is $\dot{X}_w$, can be calculated from eqn. (82) using the commutator product.

It can be shown that $\dot{M} \wedge \widetilde{M}$ is always a bivector: From equations (76) and (71), and the linearity of the involved operations, we get

$$\Omega_w = -2\dot{M} \wedge \widetilde{M} = 2M \wedge \dot{\widetilde{M}} = 2(\widetilde{\dot{M} \wedge \widetilde{M}}) = -\widetilde{\Omega}_w \qquad (83)$$

Considering the grade-dependent sign-reversal of the reverse in $3D$ EGA, Ω_w must either be a bivector or a trivector. Since $\dot{M}$ and $\widetilde{M}$ are even-grade multivectors, their geometric product results in an even-grade multivector as well, which does not have a trivector part. Thus, Ω_w and therefore $\dot{M} \wedge \widetilde{M}$ must indeed be a bivector.

This also assures, that the grade of X and $\dot{X}$ are identical. If X is a vector then $\dot{X}$ is a vector as well. This statement is true for any grade of X as long as Ω_w is a bivector.

If X_0 is not a constant element, but a function of time, which means that X_0 is a time dependent element in the *body frame* $X_0 = X_b(t)$, we have to add another term to the derivative in eqn. (82) as follows:

$$\begin{aligned}
\dot{X}(t) &= \frac{d}{dt}(M \wedge X_b \wedge \widetilde{M}) \\
&= \dot{M} \wedge X_b \wedge \widetilde{M} + M \wedge X_b \wedge \dot{\widetilde{M}} + M \wedge \dot{X}_b \wedge \widetilde{M} \\
&= cmt(M \wedge X_b \wedge \widetilde{M}, \Omega_w) + M \wedge \dot{X}_b \wedge \widetilde{M} \\
\dot{X}_w &= cmt(X_w, \Omega_w) + (\dot{X}_b)_w \qquad (84)
\end{aligned}$$

Eqn. (84) is to be considered in the world frame. However, we can also do our derivations in a completely equivalent way using the corresponding bivector rates and rotors of the *body frame*.

Now we derive the rate of change of a point $X(t)$ in the *body frame*, again assuming first that $X(t=0) = X_0 = const.$. We can write

$$\begin{aligned}
\dot{X}(t) &= \dot{M} \wedge X_0 \wedge \widetilde{M} + M \wedge X_0 \wedge \dot{\widetilde{M}} \\
&\overset{(72)}{=} 1 \wedge \dot{M} \wedge 1 \wedge X_0 \wedge \widetilde{M} - M \wedge X_0 \wedge (\widetilde{M} \wedge \dot{M} \wedge \widetilde{M}) \\
&\overset{(70)}{=} M \wedge (\widetilde{M} \wedge \dot{M}) \wedge \widetilde{M} \wedge (M \wedge X_0 \wedge \widetilde{M}) - (M \wedge X_0 \wedge \widetilde{M}) \wedge 1 \wedge \dot{M} \wedge \widetilde{M} \\
&= \left(M \wedge (\widetilde{M} \wedge \dot{M}) \wedge \widetilde{M}\right) \wedge X(t) - X(t) \wedge \left(M \wedge (\widetilde{M} \wedge \dot{M}) \wedge \widetilde{M}\right) \\
&= \left(M \wedge (\widetilde{M} \wedge \dot{M}) \wedge \widetilde{M}\right) \wedge X(t) - X(t) \wedge \left(M \wedge (\widetilde{M} \wedge \dot{M}) \wedge \widetilde{M}\right)
\end{aligned}$$

From eqn. (80) we can conclude that $-\frac{1}{2}\Omega_b(t) = \widetilde{M} \wedge \dot{M}$, resulting in

$$\begin{aligned}
\dot{X}(t) &= \left(M \wedge (-\frac{1}{2}\Omega_b(t)) \wedge \widetilde{M}\right) \wedge X(t) - X(t) \wedge \left(M \wedge (-\frac{1}{2}\Omega_b(t)) \wedge \widetilde{M}\right) \\
&= -\frac{1}{2}\left((\Omega_b)_w \wedge X - X \wedge (\Omega_b)_w\right) \\
&= -\frac{1}{2}\left(\Omega_w \wedge X - X \wedge \Omega_w\right) \\
&= -cmt(\Omega_w, X) \\
\dot{X}_w &= cmt((X_b)_w, \Omega_w) = cmt(X_w, \Omega_w) \qquad (85)
\end{aligned}$$

with $cmt(X_w, \Omega_w)$ as the commutator product between X_w and Ω_w. This is exactly the same result as before, just based on bivector rate measured in the *body frame*.

It can be shown that $\widetilde{M} \wedge \dot{M}$ is always a bivector with the same argument as before. So we don't repeat that here.

If X_0 is not a constant element, but itself a function of time $X_b(t)$, we have to add another term to the derivative in eqn. (85) again as follows:

$$\begin{aligned}
\dot{X}(t) &= \frac{d}{dt}(M \wedge X_b \wedge \widetilde{M}) \\
&= \dot{M} \wedge X_b \wedge \widetilde{M} + M \wedge X_b \wedge \dot{\widetilde{M}} + M \wedge \dot{X}_b \wedge \widetilde{M} \\
&= cmt(M \wedge X_b \wedge \widetilde{M}, \Omega_w) + M \wedge \dot{X}_b \wedge \widetilde{M} \\
&= cmt((X_b)_w, \Omega_w) + (\dot{X}_b)_w \\
\dot{X}_w &= cmt(X_w, \Omega_w) + (\dot{X}_b)_w \qquad (86)
\end{aligned}$$

Eqn. (86) is again to be considered in the *world frame*. The result derived in the *body frame* is of course identical to the result derived in the *world frame*.

In *EGA* rotation as a Euclidean isometry can be described with sandwich products based on the geometric product. Now looking into *PGA*, Lengyel has shown in [Lengyel, 2024b] that Euclidean isometries (covering translations as well as rotations) can be described with sandwich products based on the regressive geometric product in *PGA*. If we do that, we can stay in the space $G(3,0,1)$ and do not need to switch to its dual space in order to implement these orthonormal transformations as already described above.

Now we repeat the derivation of the time-derivative $\dot{X}$ of a point $X(t)$ for the projective approach of PGA used here: We will use the regressive geometric product, and $\mathbb{1}$ as its neutral element. The derivation is then completely analogous to the EGA case with the given substitutions.

M as a motor in PGA is an orthonomal transformation and satisfies

$$M \vee \underset{\sim}{M} = \underset{\sim}{M} \vee M = \mathbb{1} \tag{87}$$

as the defining equation for a motor in PGA (remember: $M^{-1} = \underset{\sim}{M}$), leads to

$$\frac{d}{dt}(M \vee \underset{\sim}{M}) = \dot{M} \vee \underset{\sim}{M} + M \vee \dot{\underset{\sim}{M}} = 0 \tag{88}$$

It follows that $M \vee \dot{\underset{\sim}{M}} = -\dot{M} \vee \underset{\sim}{M}$, which results in

$$\dot{\underset{\sim}{M}} = -\underset{\sim}{M} \vee \dot{M} \vee \underset{\sim}{M} \tag{89}$$

Starting from $\underset{\sim}{M} \vee M = \mathbb{1}$ instead yields the same result for $\dot{\underset{\sim}{M}}$.

However, we still have to take additional aspects into account:

First, Lengyel has shown that a motor in $3D$ PGA can be created from the exponential in eqn. (68) as $M = \exp_{\vee}\left[(\frac{\delta}{2}\mathbf{1} + \frac{\phi}{2}\mathbb{1}) \vee l\right]$ where it is easy to show that the regressive geometric product of the line l, a bivector, with the scalar part yields a bivector and the same is true for the regressive geometric product between the pseudoscalar and the line. Thus, it is sufficient to consider the exponential function of a bivector argument for the derivation for $3D$ PGA. The resulting motor M will be an even-grade multivector in this case.

Second, for $2D$ PGA the motor is derived from $M = \exp_{\vee}\left[\frac{\phi}{2}P\right]$ with the unitized point P, a vector, as an argument. The resulting motor M will be an odd-grade multivector. We need to distinguish these two cases later on. However, since the equations are structurally similar (an exponential function with either a vector or a bivector as argument), we only do the derivation for the $3D$ case once and then will interpret the results for the $2D$ case separately afterwards.

World frame: From eqn. (68) we know the form of a general motor in PGA and use the ansatz for a time dependent motor $M(t)$ after the the object X was brought into its initial position by M_0 as $X_0 = M \vee X \vee \underset{\sim}{M}$. If there is no difference between world and body frames at $t = 0$ then $M_0 = \mathbb{1}$:

$$M(t) = exp_{\vee}(\frac{1}{2}B_w(t)) \vee M_0 \tag{90}$$

with $B_w(t)$ as a time-dependent bivector that models the rotational plane and the turning angle. The derivative of M with respect to time t is

$$\dot{M}(t) = \frac{1}{2}\dot{B}_w(t) \vee exp_{\vee}(\frac{1}{2}B_w(t)) \vee M_0 = \frac{1}{2}\dot{B}_w(t) \vee M(t) \tag{91}$$

where we use that the exponential of the bivector and the bivector rate commute *(tbd: show why this is the case)*. Using $\Omega_w = \Omega_w(t) = \dot{B}_w(t)$ as the bivector rate of change in the *world frame*, we get

$$\dot{M} = \frac{1}{2}\Omega_w \vee M \tag{92}$$

By multiplying this from the right with "$\vee \underset{\sim}{M}$" this can transformed to

$$\Omega_w = 2\dot{M} \vee \underset{\sim}{M} \tag{93}$$

Body frame: Here we first move the object in the *body frame* and then transform it into the in the world frame by the transformation M_0. In this case eqn. (90) becomes

$$M(t) = M_0 \vee exp_{\vee}(\frac{1}{2}B_b(t)) \tag{94}$$

which in turn restults in

$$\dot{M}_b(t) = \frac{1}{2}M_0 \vee exp_{\vee}(\frac{1}{2}B_b(t)) \vee \dot{B}_b(t) = \frac{1}{2}M(t) \vee \dot{B}_b(t) \tag{95}$$

while eqn. (92) for the bivector rate of change in the *body frame* becomes

$$\dot{M} = \frac{1}{2}M \vee \Omega_b \tag{96}$$

using $\Omega_b = \Omega_b(t) = \dot{B}_b(t)$ as the bivector rate of change in the *body frame*. This can be transformed by multiplying it from the left by "$\underset{\sim}{M} \vee$" to obtain

$$\Omega_b = 2\underset{\sim}{M} \vee \dot{M} \tag{97}$$

From equations (92) and (96) we can see how world and body frame rates are connected to each other: $\Omega_w \vee M = M \vee \Omega_b$, which can be transformed into

$$\Omega_w = M \vee \Omega_b \vee \underset{\sim}{M} \tag{98a}$$

$$\Omega_b = \underset{\sim}{M} \vee \Omega_w \vee M \tag{98b}$$

Now we derive the rate of change of a point $X(t)$, i.e. its speed $\dot{X}(t)$, in the *world frame*, first assuming that $X(t=0) = X_0 = const.$, i.e. as a constant element that is transformed by a time dependent motor $M(t)$. During the derivation we will use the associativity of the regressive geometric product and the neutral element $\mathbb{1}$ of the regressive geometric product. We can write

$$\begin{aligned}
\dot{X}(t) &= \frac{d}{dt}X(t) = \frac{d}{dt}(M \vee X_0 \vee \underset{\sim}{M}) \\
&= \dot{M} \vee X_0 \vee \underset{\sim}{M} + M \vee X_0 \vee \underset{\sim}{\dot{M}} \\
&\overset{(89)}{=} \dot{M} \vee \mathbb{1} \vee X_0 \vee \underset{\sim}{M} - M \vee X_0 \vee (\underset{\sim}{M} \vee \dot{M} \vee \underset{\sim}{M}) \\
&\overset{(87)}{=} \dot{M} \vee (\underset{\sim}{M} \vee M) \vee X_0 \vee \underset{\sim}{M} - (M \vee X_0 \vee \underset{\sim}{M}) \vee \dot{M} \vee \underset{\sim}{M} \\
&= (\dot{M} \vee \underset{\sim}{M}) \vee (M \vee X_0 \vee \underset{\sim}{M}) - (M \vee X_0 \vee \underset{\sim}{M}) \vee (\dot{M} \vee \underset{\sim}{M})
\end{aligned}$$

From eqn. (93) we can conclude that $\frac{1}{2}\Omega_w(t) = \dot{M} \vee \underset{\sim}{M}$, resulting in

$$\begin{aligned}
\dot{X}(t) &= (\frac{1}{2}\Omega_w(t)) \vee X_w(t) - X_w(t) \vee (\frac{1}{2}\Omega_w(t)) \\
&= \frac{1}{2}(\Omega_w \vee X_w - X_w \vee \Omega_w) \\
\dot{X}_w &= \mathrm{rcmt}(\Omega_w, X_w) = -\mathrm{rcmt}(X_w, \Omega_w)
\end{aligned} \tag{99}$$

with $\mathrm{rcmt}(\Omega_w, X_w)$ as the regressive commutator product between Ω_w and X_w. The resulting minus sign in eqn. (99) compared to eqn. (82) is the only difference between PGA and EGA. This is caused by the different ansatz for the motors comparing equations (64) and (68).

It can be shown that $\dot{M} \vee \underset{\sim}{M}$ is either always vector or a bivector: From equations (93) and (88), and the linearity of the involved operations, we get

$$\Omega_w = 2\dot{M} \vee \underset{\sim}{M} = -2M \vee \underset{\sim}{\dot{M}} = -2(\underset{\sim}{\dot{M} \vee \underset{\sim}{M}}) = -\underset{\sim}{\Omega}_w \tag{100}$$

Considering the grade-dependent sign-reversal of the regressive reverse in $3D$ PGA, Ω_w must either be a vector or a bivector (see table 4 at the end of section 1.2). Since $\dot{M}$ and $\underset{\sim}{M}$ are even-grade multivectors, their regressive geometric product results in an even-grade multivector as well, which does not have a vector part. Thus, Ω_w and $\dot{M} \vee \underset{\sim}{M}$ must indeed be a bivector in $3D$ PGA.

However, in $2D$ PGA due to the grade-dependent sign-reversal of the regressive reverse, Ω_w must either be a scalar or a vector (see table 8 at the end of

section 1.2). Since $\dot{M}$ and $\underset{\sim}{M}$ are odd-grade multivectors, their regressive geometric product results in an odd-grade multivector as well, which does not have a scalar part. Thus, Ω_w and $\dot{M} \vee \underset{\sim}{M}$ must indeed be a vector in $2D\ PGA$.

If X_0 is not a constant element, but a function of time, which means that X_0 is a time dependent element in the *body frame* $X_0 = X_b(t)$, we have to add another term to the derivative in eqn. (99) as follows:

$$\begin{aligned}
\dot{X}(t) &= \frac{d}{dt}(M \vee X_b \vee \underset{\sim}{M}) \\
&= \dot{M} \vee X_b \vee \underset{\sim}{M} + M \vee X_b \vee \dot{\underset{\sim}{M}} + M \vee \dot{X}_b \vee \underset{\sim}{M} \\
&= -\mathrm{rcmt}(M \vee X_b \vee \underset{\sim}{M}, \Omega_w) + M \vee \dot{X}_b \vee \underset{\sim}{M} \\
\dot{X}_w &= -\mathrm{rcmt}(X_w, \Omega_w) + (\dot{X}_b)_w = \mathrm{rcmt}(\Omega_w, X_w) + (\dot{X}_b)_w \qquad (101)
\end{aligned}$$

Eqn. (101) is to be considered in the world frame. However, we can also do our derivations in a completely equivalent way using the corresponding bivector rates and rotors of the *body frame*.

The derivation in the *body frame* does not bring new insights so we don't reproduce that here for PGA again.

The final step before we move on to modelling physics is to show how points are moved in PGA using the approach described above.

In $3D$ PGA every bivector $B = B(s, \phi)$ or every bivector rate $\dot{B} = \Omega = \Omega(v, \omega)$ has its basis components

$$\Omega_{3D} = \underbrace{\Omega_{vx}\boldsymbol{e}_{41} + \Omega_{vy}\boldsymbol{e}_{42} + \Omega_{vz}\boldsymbol{e}_{43}}_{\text{weight}(\Omega)=\Omega_\circ} + \underbrace{\Omega_{mx}\boldsymbol{e}_{23} + \Omega_{my}\boldsymbol{e}_{31} + \Omega_{mz}\boldsymbol{e}_{12}}_{\text{bulk}(\Omega)=\Omega_\bullet} \tag{102}$$

where the weight part $\Omega_\circ$ carries rotational information (direction of rotation and rotational speed) and the bulk part $\Omega_\bullet$ carries translational information (positional information, orientaton of translation).

In general, the weight part only contains components that do contain a factor of $\boldsymbol{e}_4$. It has *directional* information on Ω. If there is zero weight, the object is contained in the horizon (H_{3dp}). The bulk part only has components that do not contain a factor of $\boldsymbol{e}_4$. It has *positional* information on Ω. If there is zero bulk, the object contains the origin (O_{3dp}).

We can either model a pure translation, which will only be visible in the bulk part of Ω, with the weight part being zero in that case, or we can model a pure rotation around a fixed (non-ideal) line l. In that case the positional information of the fixed line of rotation becomes encoded in the bulk part of Ω and the directional information on the rotational speed vector ω is encoded in the weight part of Ω.

A mixture of a translation and a rotation would either be equivalent to a rotation around another fixed line shifted by the translational part or, in case of translation along the axis of rotation, a screw motion around and along that axis. It is sufficient to look at pure translation and pure rotation around a fixed line as separate cases. This separation is also important for defining the exponential function with respect to the regressive geometric product that converts the bivector B to the respective motor M: $M(t) = exp_{\wedge}(B(t)) = exp_{\wedge}(\Omega t)$.

The total bivector rate Ω can be split into

$$\Omega = \Omega_\circ + \Omega_\bullet = \Omega_{rot} + \Omega_{tra} \tag{103}$$

where eqn. (103) is called the "Euclidean split" of the bivector rate Ω because it separates the rotational and translational aspects of a moving object.

A pure rotation around an axis $\vec{\omega} = \omega$ can be modelled by a fixed (unitized) line that is scaled by the angular speed of rotation, i.e. by the bulk norm $\|\omega\|_\bullet$. If we have an arbitrary rotation axis we can either wedge a unitized point $Q = O_{3dp} + q = \boldsymbol{e}_4 + q$ that is contained in the rotational axis with the rotational speed ω, or create a unitized line $\hat{\omega}$ in the direction of the rotation vector ω that is later scaled with the rotational speed such that the bivector Ω_{rot} becomes

$$\begin{aligned} \Omega_{rot} &= Q \wedge \omega = \boldsymbol{e}_4 \wedge \omega + q \wedge \omega = \hat{\omega}\, \|\omega\|_\bullet \\ \Omega_{rot} &= \|\omega\|_\bullet \begin{pmatrix} \hat{\omega}_x \\ \hat{\omega}_y \\ \hat{\omega}_z \\ q_y\hat{\omega}_z - q_z\hat{\omega}_y \\ q_z\hat{\omega}_x - q_x\hat{\omega}_z \\ q_x\hat{\omega}_y - q_y\hat{\omega}_x \end{pmatrix} \end{aligned} \tag{104}$$

where $\hat{\omega}$ is the bulk normalized rotation vector $\hat{\omega} = \omega/\|\omega\|_\bullet$. Thus, Ω_{rot} directly represents the normalized (non-ideal) line of rotation. It is a line of fixed points not changing due to the rotation, scaled with the angular speed of rotation $\|\omega\|_\bullet$.

Of course, Ω_{rot} could also be expressed directly by the plane of rotation pl (a trivector) when the magnitude of the plane encodes the angular speed $\|\omega\|_\bullet$. Considering that the left weight dual of the plane $pl_\star$ provides the normal vector of the plane including its magnitude, eqn. (104) can be written in the equivalent form directly in terms of the plane of rotation pl

$$\Omega_{rot} = Q \wedge pl_\star \tag{105}$$

For modelling translational speed $\vec{v} = v$ in $3D$ we need a bivector rate Ω_{tra} that does not contain a factor of $\boldsymbol{e}_4$. This can be achieved by encoding the speed of translation in the normal vector n_v of a plane and then intersecting this plane with the horizon $\overline{\boldsymbol{e}_4} = \overline{O_{3dp}} = H_{3dp}$. The plane with the corresponding attitude can be encoded by using the right bulk dual $v^\star$ to model the plane (a trivector) with the bulk norm $\|v\|_\bullet$ as magnitude. It contains the origin O_{3dp} and results in a bivector rate Ω_{tra} of the translation as

$$\Omega_{tra} = v^\star \vee \overline{\boldsymbol{e}_4} = v^\star \vee H_{3dp} = att(v^\star) = (0, 0, 0, v_x, v_y, v_z) \tag{106}$$

For the case of screw motion we have the special case that the translational motion is directed in the same direction as the rotation axis, i.e. we have a special Ω_{tra}

$$\Omega_{tra,screw} = \frac{\delta}{\|\omega\|_\bullet} att(\omega^\star) = \delta\,(0, 0, 0, \hat{\omega}_x, \hat{\omega}_y, \hat{\omega}_z) \tag{107}$$

where $\hat{\omega}$ is the bulk normalized vector ω, which has magnitude one. This vector is scaled with the translation distance δ resulting in Ω_{screw} of

$$\Omega_{screw} = \Omega_{rot} + \Omega_{tra,screw} = \|\omega\|_{\bullet} \begin{pmatrix} \hat{\omega}_x \\ \hat{\omega}_y \\ \hat{\omega}_z \\ q_y\hat{\omega}_z - q_z\hat{\omega}_y + \delta\hat{\omega}_x \\ q_z\hat{\omega}_x - q_x\hat{\omega}_z + \delta\hat{\omega}_y \\ q_x\hat{\omega}_y - q_y\hat{\omega}_x + \delta\hat{\omega}_z \end{pmatrix} \tag{108}$$

The components of Ω_{rot}, Ω_{tra} and Ω_{screw} are relevant for the exponential function that transforms our bivector, representing the intended motion, into a motor that transforms points accordingly.

Using for example eqn. (99) and the Euclidean split as defined in eqn. (103) we can write

$$\begin{aligned} \dot{X}_w &= \mathrm{rcmt}(\Omega_w, X_w) = \dot{x}_w \\ &= \mathrm{rcmt}(\Omega_{rot} + \Omega_{tra}, X_w) \\ \dot{x}_w &= \mathrm{rcmt}(\Omega_{rot}, X_w) + \mathrm{rcmt}(\Omega_{tra}, X_w) && \text{(109a)} \\ \dot{x}_w &= v_{rot} + v_{tra} && \text{(109b)} \end{aligned}$$

to calculate the combined speed $\dot{x}_w$ from rotational and translational effects at a position X_w.

In $2D$ PGA the equations are very similar, but the rate of change is not a rate of change bivector, but is it a rate of change vector, since the rotation is not around a fixed axis, as in $3D$, but it is around a fixed point. The translation is characterized by a $2D$ vector, whereas the rotation rate can be characterized by a single scalar value and a fixed point Q around with the rotation occurs. To show that we still get comparable equations to the $3D$ case, we will also use $\Omega = \Omega(v, \omega)$ for the rate of change. In $2D$ PGA has its basis components

$$\Omega_{2D} = \underbrace{\Omega_x \boldsymbol{e}_1 \quad + \quad \Omega_y \boldsymbol{e}_2}_{\text{bulk}(\Omega)=\Omega_\bullet} \quad + \quad \underbrace{\Omega_z \boldsymbol{e}_3}_{\text{weight}(\Omega)=\Omega_\circ} \tag{110}$$

where the weight part $\Omega_\circ$ carries rotational information and the bulk part $\Omega_\bullet$ carries translational information. In general, the weight part only contains components that do contain a factor of $\boldsymbol{e}_3$ (which here takes a role equivalent to $\boldsymbol{e}_4$ in the $3D$ case).

The total vector rate of change Ω can be split into

$$\Omega = \Omega_\bullet + \Omega_\circ = \Omega_{tra} + \Omega_{rot} \tag{111}$$

where eqn. (111) is called the "Euclidean split" of the vector rate of change Ω separating the translational and rotational aspects of a moving object.

A simple rotation in $2D$ with an angular speed ω (a simple scalar in this case) around the unitized point $Q = O_{2dp} + q = \boldsymbol{e}_3 + q$ can be modelled by multiplying the point Q with ω using the wedge product to generate the vector Ω_{rot}.

$$\Omega_{rot} = Q \wedge \omega = \boldsymbol{e}_3 \wedge \omega + q \wedge \omega = (0, 0, 1)\,\omega + (q_x, q_y, 0)\,\omega = (q_x, q_y, 1)\,\omega \tag{112}$$

For modelling translational speed $\vec{v} = v$ in $2D$ we need a vector rate of change Ω_{tra} that does not contain a factor of $\boldsymbol{e}_3$. To show this we use exactly the same equations as in the $3D$ case by encoding the speed of translation in the normal vector n_v of a line and then intersecting this line with the horizon $\overline{\boldsymbol{e}_3} = \overline{O_{2dp}} = H_{2dp}$. The double reflection on parallel lines that effectively creates the translation, necessitates the need for encoding the tanslation speed in a vector pointing normal to the translation direction. The line with the corresponding attitude can be encoded by using the right bulk dual $v^\star$ to model the line (a bivector) with the bulk norm $\|v\|_\bullet$ as magnitude. The vector rate of change Ω_{tra} of a pure translation is then given as

$$\Omega_{tra} = v^\star \vee \overline{\boldsymbol{e}_3} = v^\star \vee H_{3dp} = att(v^\star) = (-v_y, v_x, 0) \tag{113}$$

where the direction of the movement is encoded as the normal pointing towards the point at infinity which is the“rotation center at infinity” of the modelled translation.

We can either model a pure translation, which will only be visible in the bulk part of Ω, with the weight part being zero in that case, or we can model a pure rotation around a fixed (non-ideal) point Q. In that case the positional information of the fixed point becomes encoded in the bulk part of Ω and the rotational speed ω is encoded in the weight part of Ω. A mixture of a translation and a rotation would be equivalent to a rotation around another fixed point shifted by the translational part. So it is sufficient to look at pure translation and pure rotation around a fixed point as separate cases.

This separation is also important for defining the exponential function with respect to the regressive geometric product that converts the bivector B to the respective motor M: $M(t) = exp_{\wedge}(B(t)) = exp_{\wedge}(\Omega t)$ that enables us to actually transform points according to the vector rates of change as defined above.

1.4 Physics modelling

The key idea is to represent physical entities using objects represented in the underlying algebra. They have a geometric interpretation and can be manipulated by applying algebraic operations. This is possible since the algebraic objects we use in geometric algebra, especially the blades resulting from outer products between vectors, represent subspaces of the overall space and thus allow for direct algebraic manipulation. At the same time, they represent or can be attributed to objects of geometric or physical meaning. This enables implementing geometrically and physically meaningful manipulations by applying algebraic operations on those subspaces or between them.

Euclidean geometric algebra (EGA) is limited to objects that contain the origin. Thus, it can be used to directly model vectors (i.e. directions), lines, and planes through the orgin. In addition, it can model complex numbers, quaternions and thus rotations in $2D$ and $3D$ well. To realize them, it uses sandwich products based on the geometric product for calculating the transformation resulting from two consequtive reflections either across lines ($2D$ case) or across planes ($3D$ case). The resulting transformations are rotations. The advantage of using geometric algebra to model those rotations is the intuitive modelling, due to geometric meaning of the objects and the fact that the sandwich products can be used to transform different objects, like vectors, bivectors or rotation operators using the same sandwich product formulas in a unified fashion regardless of the type of object to be transformed. Despite those limitations EGA is already a very powerful tool to express physical ideas as shown by David Hestenes in [Hestenes, 1986]. In this work he reformulates classical mechanics using geometric algebra mainly with EGA as a tool. But he is also showing that even more is possible, e.g. for formulating the motion of rigid bodies, clearly pointing to PGA in his chapter 7 on rigid body mechanics, even if he does not call that PGA yet. This is already looking extremely similar to what is shown later in this section. His book is a rich source of applications of geometric algebra for many application areas of mechanics.

Projective geometric algebra (PGA) is indeed capable to model objects that do not necessarily contain the origin. Thus it can be used to directly model points, directions, lines, and planes that may or may not contain the origin. In contrast to EGA, translations can additionally be implemented in PGA directly as orthogonal transformation using a sandwich product. The operation is realized by two consequtive reflections across parallel lines ($2D$) or planes ($3D$). It is based on the regressive form of the geometric product.

The reason for this deviation from EGA, which uses the geometric product for that purpose, is that the Euclidean isometries (rotation, translation, and screw motion) can only be performed in a space that is dual to the space we use for modelling our projective objects (for a more in depth explanation see [Lengyel, 2024b]). The use of the regressive geometric product on these objects is equivalent to first transforming the object into dual space by applying the complement operation, then using the geometric product in that space for the intended transformation and finally transforming the resulting transformed object back into the original space, by applying the inverse complement operation. Thus the sandwich product transformation formulas in PGA only differ from the formulas used in EGA by using the regressive geometric product instead of the geometric product, but are otherwise the same.

The additional degree of freedom in PGA is possible due to embedding the Euclidean space of dimension n into a projective space of dimension $n+1$, e.g. two-dimensional Euclidean space is modelled by embedding it in a three-dimensional projective space, and three-dimensional Euclidean space is modelled in a four-dimensional projective space. In the $n+1$-dimensional projective space all algebraic objects contain the origin (exactly as in EGA), but in the modelled n-dimensional Euclidean space the objects do not need to contain the origin any longer. They can be located anywhere in the modelled space including positions at infinity. This can be realized in projective geometry, since objects of dimension $n+1$ are projected into the modelled space resulting in objects of dimension n after the projection. A projective point (which models a scalar or $0D$-object) is a vector representing a ray through the origin in the embedding space (a $1D$-object), and a $1D$-object, like a line in modelled space, is created by a bivector in the embedding projective space (a $2D$-object), and so on for higher-grade objects.

Due to its larger expressiveness PGA is suitable to model physics of mass points and rigid bodies including all isometries. For example, it can model position, velocity, acceleration, force, torque, momentum, angular momentum, etc., and thus it can be used to describe kinematics as well as dynamics of mass points and rigid bodies. The approach used here is inspired by [Dorst and De Keninck, 2022] and [Dorst and De Keninck, 2023], but it does not use the plane-based ansatz with planes as vectors, which are based on dual representations of the objects to be modelled. It rather uses the approach as described by [Lengyel, 2024b] using projective geometric algebra based on direct representations of points, lines and planes. Lengyel also gives reasons for choosing the latter approach and explains the background in more detail compared to the short indications given here.

Objects modelled in a projective space can be multiplied by a scalar value without changing the *geometric* meaninig of the object in the modelled Euclidean space, i.e. its interpretation after the projection. Equality of modelled geometric objects is thus often only defined up to a scalar factor. This kind of equality up to a scalar factor is defined as congruence (see e.g. [Browne, 2012]). Therefore, it is important for modelling of physical facts that unitization is used consciously to avoid false interpretations. For example, a heavy point can be modelled by multiplying a *unitized* point with the scalar value of its mass, but of course the multiplication with m must happen after unitization in order to keep the scalar factor m in the calculation. Other geometric objects need to be unitized in order to focus on the intended geometric operation and avoid additional scaling, i.e. the rotation operators. They must be unitized before use as well. Thus it is advisable to think through the geometric operations and their physical interpretation well to avoid suprises and to be consistent.

[Lengyel, 2024a] points out additional potential mathematical inconsistencies in applications of geometric algebra as used up-to-now quite often. Further hints on mathematical inconsistencies in some GA approaches can be found as well in [Kritchevsky, 2024]. The goal of the this work is to be as consistent and mathematically rigorous as possible implementing GA functionality, and at the same time providing an intuitively applicable framework for modelling physics and applications.

1.4.1 Modelling force and torque

[Browne, 2012] (see sections 4.1-4.5) is a very good source showing how to use Grassmann algebra and projective geometry in order to model forces and moments. Classical approaches based on vector algebra only model forces by providing vectors of adequate magnitude and direction to represent the force. However, the physically very meaningful "line of action", which determines which torque is created by the force, is not modelled mathematically in the classical approach at all. It rather needs to be derived from the description of the context in order to know where the force is actually acting. Thus force and torqure are not directly interlinked in the classical description, but must be modelled as separate physical effects.

This changes in *PGA*. Any force can be directly modelled by its line of action, which encodes the force *and* the torque generated by the force at the

origin within one algebraic object. A force f is modelled by a line F. If f is the force vector of corresponding magnitude and direction, and Q is any unitized projective point on its line of action then F is an algebraic object representing the force f as well as the resulting line of action directly:

$$F = Q \wedge f \tag{114}$$

The resulting bivector F represents a line, a geometric object which contains all physically relevant information of the force f and the torque generated by f. For that reason [Dorst and De Keninck, 2023] call this object "forque". It can directly be used to recover the force vector and to calculate the resulting free torque caused by the force either at the origin or at any point in space.

But first, let's have a closer look how this is accomplished. We can separate F into its components. Each unitized projective point Q is the sum of the origin O and a vector q parallel to the projective plane (in $2D$) or projective space (in $3D$) pointing from O to Q, such that $Q = O + q$ ("projective point split"). Thus F can be split into two bivector components

$$F = Q \wedge f = (O + q) \wedge f = \underbrace{O \wedge f}_{=\text{force}} + \underbrace{q \wedge f}_{=torque} = F_\circ + F_\bullet \tag{115}$$

where the first term encodes the force within the weight components of the bivector, while the second term encodes the resulting free torque relative to the origin caused by the action of f at Q – or alternatively anywhere along the line of action – within the bulk components of the bivector.

To see this even more clearly, let's look at the component equations of both terms written as bivectors for the case of $2D$ PGA (the corresponding basis bivectors are shown on the right-hand side in square brackets for reference):

$$O \wedge f = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \wedge \begin{pmatrix} f_x \\ f_y \\ 0 \end{pmatrix} = \begin{pmatrix} f_x \\ f_y \\ 0 \end{pmatrix} \qquad \begin{bmatrix} \boldsymbol{e}_{31} \\ \boldsymbol{e}_{32} \\ \boldsymbol{e}_{12} \end{bmatrix} \tag{116a}$$

$$q \wedge f = \begin{pmatrix} q_x \\ q_y \\ 0 \end{pmatrix} \wedge \begin{pmatrix} f_x \\ f_y \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ q_x f_y - q_y f_x \end{pmatrix} \qquad \begin{bmatrix} \boldsymbol{e}_{31} \\ \boldsymbol{e}_{32} \\ \boldsymbol{e}_{12} \end{bmatrix} \tag{116b}$$

where equation (116a) shows how the force is encoded in the first two components of the bivector and equation (116b) shows how the torque is encoded in the third component of the bivector F.

For $3D$ PGA the encoding is similar, but now requires six bivector components instead of three (the corresponding basis bivectors are shown on the right-hand side in square brackets for reference again):

$$O \wedge f = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} \wedge \begin{pmatrix} f_x \\ f_y \\ f_z \\ 0 \end{pmatrix} = \begin{pmatrix} f_x \\ f_y \\ f_z \\ 0 \\ 0 \\ 0 \end{pmatrix} \qquad \begin{bmatrix} \boldsymbol{e}_{41} \\ \boldsymbol{e}_{42} \\ \boldsymbol{e}_{43} \\ \boldsymbol{e}_{23} \\ \boldsymbol{e}_{31} \\ \boldsymbol{e}_{12} \end{bmatrix} \tag{117a}$$

$$q \wedge f = \begin{pmatrix} q_x \\ q_y \\ q_z \\ 0 \end{pmatrix} \wedge \begin{pmatrix} f_x \\ f_y \\ f_z \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ q_y f_z - q_z f_y \\ q_z f_x - q_x f_z \\ q_x f_y - q_y f_x \end{pmatrix} \qquad \begin{bmatrix} \boldsymbol{e}_{41} \\ \boldsymbol{e}_{42} \\ \boldsymbol{e}_{43} \\ \boldsymbol{e}_{23} \\ \boldsymbol{e}_{31} \\ \boldsymbol{e}_{12} \end{bmatrix} \tag{117b}$$

The first three bivector components modell the force while the last three components model the torque. The force vector f itself can be retrieved from F by using the attitude function $att()$:

$$f = att(F) = F \vee \overline{\boldsymbol{e}_n} \tag{118}$$

with $\boldsymbol{e}_n$ denoting the basis vector of the projective dimension that squares to zero in the respective algebra ($\boldsymbol{e}_3$ for $2D$ and $\boldsymbol{e}_4$ for $3D$). The torque generated relative to the origin O is encoded in the bulk of the bivector:

$$M_O = bulk(F) = F_\bullet \tag{119}$$

while the resulting torque from the application of f at any given point R in space can be retrieved from F, either using any known point Q on F[5], or alternatively by projecting R onto F and using the perpendicular distance vector $d_\perp$ from R towards the line F in order to calculate the resulting free torque $M_F(R)$ at the position R as

$$M_F(R) = [(Q - R) \wedge f]_\bullet = [(Q - R) \wedge att(F)]_\bullet \tag{120a}$$

$$M_F(R) = [d_\perp \wedge f]_\bullet = [(F \vee (R \wedge F^\star) - R) \wedge att(F)]_\bullet \tag{120b}$$

where equation (120a) is the formula with the least computational effort using a known point Q on F, while equation (120b) uses the othogonal projection of R onto F to calculate the perpendicular distance vector $d_\perp$.

[5] Which is always available from the function $sup(F)$. It provides the support point on F with the shortest distance to the origin O by orthogonally projecting the origin onto F.

The resulting total force and torque acting on an object are given by:

$$F_{total} = \sum_i F_i \tag{121}$$

This resulting sum will be used later for connecting kinematics and dynamics.

Things to remember:

- Neither a vector nor a bivector are located anywhere. They can be moved around freely. A line however represents a bound vector that is bound by its line. Thus the vector can only be moved along its line of action without changing the geometric and physical meaning (see [Browne, 2012], p. 169).
- A free torque is modelled by a bivector with force components being zero, i.e. the resulting force vector (e.g. from summing several forces acting on an object) has no direction and zero length, but the bivector still might contain a resulting torque value.
- The torque calculated from either equation (120a) or (120b) is necessarily a free torque as can be seen from the formulas. The origin does not occur in the result anymore due to the difference of the two projective points involved.
- The sum of forces and their induced torque values can be directly created by the sum of their respective lines of action, i.e. by summing the bivectors representing those lines. The resulting lines of action contain all relevant information incl. the resulting vector sum of the forces and the resulting torque values from the summation. There is no need to calculate these separately. The result will always be a resulting bivector that potentially contains a bound vector and a resulting torque. (see [Browne, 2012], p. 176).

1.4.2 Modelling linear and angular momentum

Momentum in the classical approach is split into linear momentum $\vec{p} = m\vec{v}$ and angular momentum $\vec{L} = \vec{x} \times \vec{p} = \vec{x} \times m\vec{v}$. Both are represented as vectors, but they represent entities with different physical meaning. So they cannot be combined into one object in the classical approach, e.g. by adding them.

The *PGA* approach to momentum is to define both, the linear and the angular aspects, within one algebraic entity. This is in complete analogy to what

we have seen in the last subsection for force and torque resulting in its line of action modelled by a bivector.

The momentum P of a point mass m at a projective (unitized) position X that moves with velocity $\dot{X}$ is defined as a bivector P as

$$P = X \wedge m\dot{X} \tag{122}$$

It defines the overall momentum at position X including linear and angular momentum in the bivector $P(X)$, which can be shown by splitting the projective point X into its components $X = O + x$ ("projective point split"). In the $3D$-case with the origin $O = O_{3dp} = (0, 0, 0, 1)$ and the direction vector x, where $x = (X_x, X_y, X_z, 0)$, i.e. specifically with $X_w = 0$, which represents a direction towards a point at infinity. This results in

$$\begin{aligned} P(X) &= X \wedge m\dot{X} = (O + x) \wedge m\dot{X} \\ &= O \wedge m\dot{X} + x \wedge m\dot{X} \\ &= O \wedge m(O \dot{+} x) + x \wedge m(O \dot{+} x) \\ &= O \wedge m\dot{x} + x \wedge m\dot{x} && \text{(123a)} \\ &= \underbrace{P_\circ}_{\text{linear}} + \underbrace{P_\bullet}_{\text{angular}} && \text{(123b)} \end{aligned}$$

P defines an instantaneous momentum line where the weight part $P_\circ$ of the bivector P encodes the linear momentum and the bulk part $P_\bullet$ encodes the angular momentum with respect to the origin.[6]

The physical aspects encoded in the momentum line P can be retrieved in complete analogy to the case of force and torque in subsection 1.4.1:

The linear momentum vector p itself can be retrieved from P by using the attitude function $att()$:

$$p = mv = m\dot{x} = att(P) = P \vee \overline{\boldsymbol{e}_n} \tag{124}$$

with $\boldsymbol{e}_n$ denoting the basis vector of the projective dimension that squares to zero in the respective algebra ($\boldsymbol{e}_3$ for $2D$ and $\boldsymbol{e}_4$ for $3D$).

[6]The "projective point split" works in this way, since the origin is a destinguishable point. This is in contrast to EGA or the classical approach where the orgin is a null-vector, which has no lasting effect if it is added to another point. This is one reason why PGA can combine linear and angular aspects as separate physical effects in a single algebraic object. The other is that the required 6 degrees of freedom can be encoded in a bivector.

The angular momentum L generated relative to the origin O is encoded in the bulk of the bivector:

$$L_O = x \wedge m\dot{x} = bulk(P) = P_\bullet \tag{125}$$

while the resulting angular momentum relative to any given point R in space can be retrieved from the momentum line P by

$$L(R) = \left[(X - R) \wedge m\dot{X}\right]_\bullet = \left[(X - R) \wedge att(P)\right]_\bullet \tag{126}$$

using X as a known point on P. This provides the resulting angular momentum $L(R)$ with respect to R. The momentum line encodes the resulting momentum including its linear and angular aspects relative to any arbitrarily chosen point.

The equations (124) and (125) given above permit to separate the linear and angular effects as in the classical approach. However, these aspects need not be separated at all and can be calculated together in PGA.

Using eqn. (122) and the definition $X = O + x$ with X as a unitized projective point, O as the chosen origin and x as the direction vector pointing from O to X we can write

$$P_O(X) = X \wedge m\dot{X} = (O + x) \wedge m(\dot{O + x}) \tag{127}$$

Compared to eqn. (122) an index O was added to P, since the formula (127) seems to confirm that the momentum depends on the choice of the origin. Now it is important to demonstrate that this is *not* the case. We are going to demonstrate that the described physics does not depend on the chosen origin and not on the resulting coordinates used for its description.

First approach: Use a transformation of the standpoint from which we describe the physical situation without actually moving the coordinate system (we still want use the chosen coordinate system to describe the vectors and bivectors used): We go to a virtual origin $O' = Y$, but describe the situation in the same coordinate system. This results in describing the origin as $O = Y - y$ as seen from Y. Inserting that definition into eqn. (127) and using the geometric vector identity

$$X = Y + (X - Y) = Y + (O + x - (O + y)) = Y + (x - y) \tag{128}$$

we express $P(X)$ as viewed from Y by reformulating the equation for P

relative to the origin O

$$\begin{aligned} P_O(X) &= X \wedge m\dot{X} = (O + x) \wedge m\dot{(O + x)} \\ &= (Y - y + x) \wedge m\dot{(Y - y + x)} \\ &= (Y + (x - y)) \wedge m\dot{(Y + (x - y))} = P_Y(X) \end{aligned} \tag{129}$$

where the last equation obviously is the same as eqn. (127), but here Y takes the role of O and $(x - y)$ takes the role of the vector x in the old description. The dot used for the time-derivative refers to the whole expression in parentheses. So the same physical situation was described from another viewpoint. However, the algebraic object P is unchanged, i.e. $P_O(X) = P_Y(X)$. This clearly shows that P does not depend on the chosen origin and that eqn. (127) is indeed a coordinate free definition.

Second approach: This one is not so explicit on being independent of the origin. We just express X by Y using eqn. (127) and (128) directly to write

$$\begin{aligned} P_O(X) &= X \wedge m\dot{X} \\ &= (Y + (X - Y)) \wedge m\dot{(Y + (X - Y))} \\ &= (Y + (x - y)) \wedge m\dot{(Y + (x - y))} = P_Y(X) \end{aligned} \tag{130}$$

We can again see that this is structurally identical to eqn. (127) just seen from Y instead from O.

The total momentum of a set of mass points $m_i X_i$ is defined as the sum of all momentum blades.

$$P_{total} = \sum_i P_i \tag{131}$$

P_{total} will always be a momentum blade in $2D\ PGA$, while in $3D\ PGA$ (a four-dimensional algebra) it could also result in a general bivector that might not be factorizable into a blade (see [Browne, 2012]). That would be the case, if $P \wedge P \neq 0\ \mathbb{1}$, i.e. when the result is a pseudoscalar that is not equal to a zero multiple of the pseudoscalar $\boldsymbol{e}_{1234}$. This test can always be used to determine whether a bivector is a blade in $3D\ PGA$.

Using discrete points $X_{w,i}$ of the world frame indexed by i that can be moved by a motor $M(t) = exp_{\vee}[\frac{1}{2}B_w(t)] \vee M_0$ in the world frame as by eqn. (69) $X_{w,i} = M \vee X_{0,i} \vee \underset{\sim}{M}$ results in a local speed of $\dot{X}_{w,i} = \mathrm{rcmt}(\Omega_w, X_{w,i})$ as shown in eqn. (99). Therefore, we can write the total momentum P_w in the world

frame as

$$\begin{aligned} P_w &= \sum_i P_i \\ &= \sum_i X_i \wedge m_i \dot{X}_i \\ &= \sum_i X_i \wedge m_i \,\mathrm{rcmt}(\Omega_w, X_i) \\ P_w &\equiv I_w[\Omega_w] \end{aligned} \tag{132}$$

where eqn. (132) is a linear function of Ω_w that maps the bivector rate in the world frame Ω_w to the total momentum P_w. It is a function that takes a bivector as an argument and returns a bivector (the "intertia map" I_w). It describes the influence of the mass distribution of the points on the movement via the combined Newton/Euler law

$$\dot{P}_w = F_w \tag{133}$$

(for the total force in the *world frame* $F_{total} = F_w$ see eqn. (121)). If we go from discrete mass points to a continuous mass distribution, the sum has to be replaced by an integral over $dm = \rho dV$.

The equation $\dot{P}_w = X_w \wedge m\ddot{X}_w = F_w$ cannot be integrated directly since the second derivative $\ddot{X}_w$ cannot be isolated on the left-hand side of the equation as required for direct integration. We introduce another variable that can be isolated, enabling us to solve the ODE. For this purpose we use the rate of change Ω (a vector in $2D$ and a bivector in $3D$). The integration of that variable will then allow for computation of position, speed, etc. as a separate step.

Calculating the derivative $\dot{P}_w$ in the world frame in general yields two terms

$$\dot{P}_w = \dot{I}_w[\Omega_w] + I_w[\dot{\Omega}_w] \tag{134}$$

since in the world frame the inertia $I_w[\Omega_w]$ map will change over time due to moving points as well as due to the rate of change Ω_w. Therefore, it is reasonable to calculate the inertia map in the body frame, since it is constant in that frame, if the body frame is chosen to be fixed to the modelled rigid object. This avoids the effort to additionally calculate the time derivative of the inertia map (the first term on the right-hand side above).

If we want to express the physical problem in the body frame, we have to transform the components of eqn. (132) accordingly. Using eqn. (98b), i.e.

$I_b = \underset{\sim}{M} \vee I_w \vee M$ and eqn. (98a) $\Omega_w = M \vee \Omega_b \vee \underset{\sim}{M}$ then results in

$$\begin{aligned} P_w &= I_w[\Omega_w] \\ \underset{\sim}{M} \vee P_w \vee M &= \underset{\sim}{M} \vee I_w[\underset{\sim}{M} \vee \Omega_w \vee M] \vee M \\ P_b &= I_b[\Omega_b] \end{aligned}$$

which finally results in an expression for the inertia map

$$I_b[\Omega_b] \equiv \underset{\sim}{M} \vee I_w[M \vee \Omega_b \vee \underset{\sim}{M}] \vee M \tag{136}$$

The same transformation must be applied to F_w for the right-hand side of the differential equation $\dot{P}_w = F_w$. Therefore, we get in the body frame $F_b = \underset{\sim}{M} \vee F_w \vee M$, resulting in $\dot{P}_b = F_b$.

The time derivative in the body frame is simpler, as long as we restrict ourselves to rigid bodies, resulting in only one term, since the rate of change Ω_b is the only time dependent function in the body frame

$$\dot{P}_b = I_b[\dot{\Omega}_b] \tag{137}$$

otherwise the term $\dot{I}_b[\Omega_b]$ would have to be covered as well, as in eqn. (134).

The mapping to the classical intertia can be done when splitting Ω_b relative to the centroid of the body with a projective point split on Ω_b and on $I_b[\Omega_b]$ as shown by [Dorst and De Keninck, 2023] on page 25f and adjusted to our approach using regressive geometric products.

Now we introduce some typically used reference points to describe kinemetics via the bivector rate of change Ω and dynamics via the total momentum P:

- *Fixed body refence point* O_b used as origin of the body frame. When the body is moved by the motor M it will move in the world frame to $O_{b,w} = M \vee O_b \vee \underset{\sim}{M}$.
- *Relative locations* r_i *versus* O_b are positions in the body frame. Their projective counterpart in the body frame is $R_i = O_b + r_i$. When transformed to the world frame we get X_i in the world frame by the usual transformation $X_i = M \vee R_i \vee \underset{\sim}{M} = M \vee (O_b + r_i) \vee \underset{\sim}{M}$.
- *Center of mass* in the body frame is chosen to be located at O_b, such that $\sum_i m_i r_i = \vec{0}$. Thus the origin of the body frame is located at the center of mass. The center of mass in the world frame C_w then is $C_w \equiv \sum_i m_i X_i / \sum_i m_i = \sum_i m_i (M \vee (O_b + r_i) \vee \underset{\sim}{M}) / \sum_i m_i = M \vee O_b \vee \underset{\sim}{M}$.

The last missing piece for dynamics is the time derivative of P_w expressed as time rate of change Ω_b of the inertia in the body system. Applying eqn. (99) in the body frame, which is not only valid for any X, but for any object moved by a motor M, we get

$$\frac{d}{dt}P_b = \frac{d}{dt}I_b[\Omega_b] = \mathrm{rcmt}(\Omega_b, I_b[\Omega_b]) + I_b[\dot{\Omega}_b] \tag{138}$$

where the last term arises since Ω_b is also time-dependent. Solving eqn. (133), but in the body frame, we get

$$F_b = \dot{P}_b = \mathrm{rcmt}(\Omega_b, I_b[\Omega_b]) + I_b[\dot{\Omega}_b] \tag{139}$$

resolving this for $\dot{\Omega}_b$ we get

$$I_b[\dot{\Omega}_b] = F_b - \mathrm{rcmt}(\Omega_b, I_b[\Omega_b]) \tag{140}$$

$$\dot{\Omega}_b = I_b^{-1}\left[F_b - \mathrm{rcmt}(\Omega_b, I_b[\Omega_b])\right] \tag{141}$$

Together with the ansatz in the body system eqn. (94) that can be used to calculate M(t) when we know Ω_b at a given time t from

$$M(t) = M_0 \vee exp_\vee(\frac{1}{2}B_b(t)) = M_0 \vee exp_\vee(\frac{1}{2}\Omega_b t)$$

and the rate of change in the body frame according to eqn. (96) we get

$$\dot{\Omega}_b = I_b^{-1}\left[F_b - \mathrm{rcmt}(\Omega_b, I_b[\Omega_b])\right] \tag{142}$$

as an ODE that can be solved as initial value problem. Due to $\dot{\Omega}_b = \ddot{B}_b$ it actually is a 2nd order ODE (like the classical Newton/Euler equations that can be integrated directly from the instantaneous acceleration and the angular acceleration, respectively, in order to calculate speed and position). Here we rather integrate from $\ddot{B}_b$ in order to calculate $\dot{B}_b = \Omega_b$ and B_b. Thus we require initial values for Ω_b and B_b. The actual positions of the points of the rigid body can then be computed from eqn. (69) $X_w(t) = M(t) \vee X_0 \vee \underset{\sim}{M}(t)$. The speed of the points can be calculated either from eqn. (99) $\dot{X}_w = \mathrm{rcmt}(\Omega_w, X_w)$ or eqn. (101) $\dot{X}_w = \mathrm{rcmt}(\Omega_w, X_w) + (\dot{X}_b)_w$ – which one is applicable depends on whether the rigid body is fixed in the body frame or moves in the body frame. The expressions in the world frame can of course be expressed in the body frame as well using eqn. (98a) $\Omega_w = M \vee \Omega_b \vee \underset{\sim}{M}$ and $X_w = M \vee X_b \vee \underset{\sim}{M}$.

For that purpose we need the inertia map $I_b[\Omega_b]$ and its inverse to set up the right-hand side of the ODE system given in equations (142). We start with

the $2D$ case where the rate of change Ω_b is a vector, as shown at the end of subsection 1.3. We assume discrete points X_i and use the wedge product $X \wedge Y$ and the regressive commutator product term $m\,\mathrm{rcmt}(\Omega_b, X)$ to express the total momentum P written in component form:

$$X \wedge Y = \begin{pmatrix} X_x \\ X_y \\ X_z \end{pmatrix} \wedge \begin{pmatrix} Y_x \\ Y_y \\ Y_z \end{pmatrix} = \begin{pmatrix} X_z Y_x - X_x Y_z \\ X_z Y_y - X_y Y_z \\ X_x Y_y - X_y Y_x \end{pmatrix} \tag{143a}$$

$$m\,\mathrm{rcmt}(\Omega_b, X) = m \begin{pmatrix} \Omega_y X_z - \Omega_z X_y \\ \Omega_z X_x - \Omega_x X_z \\ 0 \end{pmatrix} \tag{143b}$$

Now the inertia map for a point X with mass m and rate of change Ω can be expressed in the $2D$ case as

$$\begin{aligned} I_{2D}[\Omega] &= X \wedge m\,\mathrm{rcmt}(\Omega, X) \\ &= m \begin{pmatrix} X_z(\Omega_y X_z - \Omega_z X_y) \\ X_z(\Omega_z X_x - \Omega_x X_z) \\ X_x(\Omega_z X_x - \Omega_x X_z) - X_y(\Omega_y X_z - \Omega_z X_y) \end{pmatrix} \end{aligned}$$

$$I_{2D}[\Omega] = m \begin{bmatrix} 0 & X_z^2 & -X_y X_z \\ -X_z^2 & 0 & X_x X_z \\ -X_x X_z & -X_y X_z & X_x^2 + X_y^2 \end{bmatrix} \begin{bmatrix} \Omega_x \\ \Omega_y \\ \Omega_z \end{bmatrix} \tag{144}$$

where eqn. (144) is linear in Ω as expected. Of course, the total inertia map needs to be summed for the contributions from each point, i.e. $I_{total} = \sum_i I_i$. Since all points of the rigid body see the same rate of change the total inertia map can be summed for all mass point contributions independent of the rate of change and the resulting total interia map can be inverted either symbolically or numerically to get its inverse I_{2D}^{-1} as needed to set up the right-hand side of the system in eqn. (142). The inertia map in $2D$ maps the rate of change vector to the momentum bivector. The inverse inertia map than maps from a momentum bivector to the rate of change vector.

Now let's do the same for the $3D$ case, where the rate of change is a bivector. The component expressions are generated from the corresponding product tables in `ga_prdxpr` and integrated into the products in the ga library.

$$X \wedge Y = \begin{pmatrix} X_x \\ X_y \\ X_z \\ X_w \end{pmatrix} \wedge \begin{pmatrix} Y_x \\ Y_y \\ Y_z \\ Y_w \end{pmatrix} = \begin{pmatrix} X_w Y_x - X_x Y_w \\ X_w Y_y - X_y Y_w \\ X_w Y_z - X_z Y_w \\ X_y Y_z - X_z Y_y \\ X_z Y_x - X_x Y_z \\ X_x Y_y - X_y Y_x \end{pmatrix} \tag{145a}$$

$$m \operatorname{rcmt}(\Omega_b, X) = m \begin{pmatrix} \Omega_{vy} X_z - \Omega_{vz} X_y + \Omega_{mx} X_w \\ \Omega_{vz} X_x - \Omega_{vx} X_z + \Omega_{my} X_w \\ \Omega_{vx} X_y - \Omega_{vy} X_x + \Omega_{mz} X_w \\ 0 \end{pmatrix} \tag{145b}$$

Now the intertia map for a point X with mass m and rate of change Ω can be expressed in the $3D$ case as

$$I_{3D}[\Omega] = X \wedge m \operatorname{rcmt}(\Omega, X) =$$

$$m \begin{pmatrix} X_w(\Omega_{vy} X_z - \Omega_{vz} X_y + \Omega_{mx} X_w) \\ X_w(\Omega_{vz} X_x - \Omega_{vx} X_z + \Omega_{my} X_w) \\ X_w(\Omega_{vx} X_y - \Omega_{vy} X_x + \Omega_{mz} X_w) \\ X_y(\Omega_{vx} X_y - \Omega_{vy} X_x + \Omega_{mz} X_w) - X_z(\Omega_{vz} X_x - \Omega_{vx} X_z + \Omega_{my} X_w) \\ X_z(\Omega_{vy} X_z - \Omega_{vz} X_y + \Omega_{mx} X_w) - X_x(\Omega_{vx} X_y - \Omega_{vy} X_x + \Omega_{mz} X_w) \\ X_x(\Omega_{vz} X_x - \Omega_{vx} X_z + \Omega_{my} X_w) - X_y(\Omega_{vy} X_z - \Omega_{vz} X_y + \Omega_{mx} X_w) \end{pmatrix} =$$

$$m \begin{bmatrix} 0 & X_z X_w & -X_y X_w & X_w^2 & 0 & 0 \\ -X_z X_w & 0 & X_x X_w & 0 & X_w^2 & 0 \\ X_y X_w & -X_x X_w & 0 & 0 & 0 & X_w^2 \\ X_y^2 + X_z^2 & -X_x X_y & -X_x X_z & 0 & -X_z X_w & X_y X_w \\ -X_x X_y & X_x^2 + X_z^2 & -X_y X_z & X_z X_w & 0 & -X_x X_w \\ -X_x X_z & -X_y X_z & X_x^2 + X_y^2 & -X_y X_w & X_x X_w & 0 \end{bmatrix} \begin{bmatrix} \Omega_{vx} \\ \Omega_{vy} \\ \Omega_{vz} \\ \Omega_{mx} \\ \Omega_{my} \\ \Omega_{mz} \end{bmatrix} \tag{146}$$

where eqn. (146) is linear in Ω as expected. Again, the total inertia map needs to be summed for the contributions from each point, i.e. $I_{total} = \sum_i I_i$. The resulting total interia map can be inverted numerically to get its inverse I_{3D}^{-1} as needed to set up the right-hand side of the system in eqn. (142). The inertia map in $3D$ maps the rate of change bivector to the momentum bivector. The inverse inertia map than maps from a momentum bivector to the corresponding rate of change bivector.

As shown in [Dorst and De Keninck, 2023] the inertia map is additive. That means that inertia maps of parts of a body consisting of several parts that move with the same bivector rate can be added to get the total inertia.

1.4.3 Solving the Newton/Euler equations numerically

In classical mechanics the Newton equation states that change of linear momentum $\vec{p} = m\vec{v}$ of a mass point of mass m moving with speed $\vec{v}$ is driven by an external force $\vec{f}$ acting on the mass. This results in $\dot{\vec{p}} = m\dot{\vec{v}} = m\ddot{\vec{x}}_{cm} = \vec{f}$ for translational motion in a system with constant mass m. The center of mass at position $\vec{x}_{cm}$ moves with the velocity $\vec{v}$ as a result.

The analogous equation for rotational motion, attributed to Euler, states that the temporal change of angular momentum $\vec{L} = \vec{x} \times \vec{p} = \vec{x} \times m\vec{v}$ is driven by an external angular momentum $\vec{M}$. $\vec{M}$ is given as $\vec{M} = \vec{r} \times \vec{f}$ with $\vec{r}$ as the vector pointing from the rotational axis to the point of action of the force $\vec{f}$. The equation for rotational motion $\dot{\vec{L}} = J\dot{\vec{\omega}} = \vec{M}$ for a rigid body with moment of inertia $J = \int_m r_\perp^2 dm = \int_V r_\perp^2 \rho dV$ with $r_\perp$ as the perpendicular distance of the mass point within the rigid body to the momentary rotational axis. The equation determines the temporal development of the orientation of the rigid body which spins with the angular speed $\|\vec{\omega}\|$ around the momentary rotation axis defined by $\vec{\omega}$ as a result.

Due to the same structure the equations can be written as one system of equations that can be solved as uncoupled equations (e.g. covering cases with either translation or rotation exclusively within the identical structure). In the $2D$ case the translational part requires two degrees of freedom (DoF) and the rotational part needs one DoF resulting in a total of three DoF. It can be written compactly as a system introducing the inertia matrix I as a linear function that maps the acceleratory degrees of freedom to the corresponding external force and torque components[7]:

$$\underbrace{\begin{bmatrix} m & 0 & 0 \\ 0 & m & 0 \\ 0 & 0 & J \end{bmatrix}}_{inertia\ I} \begin{pmatrix} \ddot{x}_x \\ \ddot{x}_y \\ \ddot{\phi} \end{pmatrix} = \begin{pmatrix} f_x \\ f_y \\ M \end{pmatrix} \tag{147}$$

[7]Please be aware, that this does not actually couple the equations! We just solve two independent equations conveniently in parallel to show the principle here. If the movement is actually coupled, there will be coupling terms to model physical requirements that will be shown later on.

Solving such a system requires to isolate the accelerations by multiplying the system from the left with the inverse of the inertia map I^{-1} resulting in

$$\begin{pmatrix} \ddot{x}_x \\ \ddot{x}_y \\ \ddot{\phi} \end{pmatrix} = \underbrace{\begin{bmatrix} \frac{1}{m} & 0 & 0 \\ 0 & \frac{1}{m} & 0 \\ 0 & 0 & \frac{1}{J} \end{bmatrix}}_{inverse\ inertia\ I^{-1}} \begin{pmatrix} f_x \\ f_y \\ M \end{pmatrix} \tag{148}$$

Both equations result in a system of ordinary differential equation (ODE) of second order that requires two initial conditions, one for the initial position $\vec{x}(t = 0) = \vec{x}_0$ and the initial speed $\vec{v}(t = 0) = \vec{v}_0$ (Newton equation) and one for the initial orientation $\phi(t = 0) = \phi_0$ and the initial angular speed $\omega(t = 0) = \omega_0$ (Euler equation) to be integrated. In the $2D$ case ϕ and ω are scalar values, while in the $3D$ case they also become vectors each representing three degrees of freedom.

To integrate the equations the system of 2nd order must be transfered into a system of 1st order. We have a 2nd order ODE for $y(t)$ with $\ddot{y} = \frac{\partial^2 y}{\partial t^2}$ of the general form

$$\ddot{\vec{y}} = f(\dot{\vec{y}}, \vec{y}, t) \tag{149}$$

The vector y has components that we index with n. We use an additional index to distinguish between the position (index 0) and the speed (index 1) of the respective DoF. We use the substitution

$$y_{n,0} = y_n \qquad \rightarrow position \tag{150a}$$
$$y_{n,1} = \dot{y}_n \qquad \rightarrow speed \tag{150b}$$

resulting in the following system of 1st order for each component n of the vector $\vec{y}$:

$$\dot{y}_{n,0} = y_{n,1} \tag{151a}$$
$$\dot{y}_{n,1} = f(y_{n,1}, y_{n,0}, t) \tag{151b}$$

If we would have to integrate for more than one point, we could easily add an additional index at the beginning to extent the system accordingly. However, for now we only focus on integrating the system for one point (our center of mass) at which we want to know position and speed as well as angular orientation and angular speed as function of time. The substitution above

transforms the system in eqn. (148) into a system of 1st order as follows:

$$\dot{x}_x = v_x \tag{152a}$$
$$\dot{v}_x = \frac{1}{m} f_x \tag{152b}$$
$$\dot{x}_y = v_y \tag{152c}$$
$$\dot{v}_y = \frac{1}{m} f_y \tag{152d}$$
$$\dot{\phi} = \omega \tag{152e}$$
$$\dot{\omega} = \frac{1}{J} M \tag{152f}$$

This 1st order ODE system $\dot{\vec{u}} = rhs(\vec{u}, t)$ can now be solved be direct integration, using e.g. a Runge-Kutta-Scheme of 4th order accuracy for precise time integration, and using the initial conditions defined above. As a result we directly get the positions and velocities as function of time.

Transition to the body frame: the coupled classical equations. The derivation above works in the inertial (lab) frame, where Newton's and Euler's equations are genuinely uncoupled. However, for a non-spherical rigid body the inertia tensor $\mathbf{I} = \mathbf{I}(t)$ rotates with the body and is therefore time-varying. Switching to a frame fixed to the body keeps $\mathbf{I} = \mathbf{I}_b$ constant – at the cost of introducing coupling terms between translation and rotation that arise from the rotation of the frame itself. This body-frame formulation is the standard choice for rigid-body simulation and is also the natural setting for the *PGA* formulation. The key differences are summarised in Table 9.

	Inertial frame	Body-fixed frame
Inertia tensor $\mathbf{I}$	Time-varying $\mathbf{I}(t)$	$\mathbf{I}_b$ constant
Coupling terms	None	$\vec{\omega} \times \vec{v}$ and $\vec{\omega} \times \mathbf{I}_b \vec{\omega}$
Practical for non-spherical bodies	Expensive	Standard choice

Table 9: Inertial frame vs. body-fixed frame: key differences for rigid-body integration.

In the body frame, the translational and rotational equations of motion become (now actually coupled, descibing translational and rotational motion at the same time):

$$m\left(\dot{\vec{v}} + \vec{\omega} \times \vec{v}\right) = \vec{F}^b \tag{153}$$
$$\mathbf{I}_b \dot{\vec{\omega}} + \vec{\omega} \times \left(\mathbf{I}_b \vec{\omega}\right) = \vec{\tau}^b \tag{154}$$

where $\vec{v}$ and $\vec{\omega}$ are the translational and angular velocity expressed in the body frame, $\mathbf{I}_b$ is the (constant) inertia tensor in the body frame, and $\vec{F}^b$, $\vec{\tau}^b$ are the external force and torque expressed in the body frame. Solving for the derivatives gives the state-space form (linear and angular accelerations isolated):

$$\dot{\vec{v}} = \frac{\vec{F}^b}{m} - \vec{\omega} \times \vec{v} \tag{155}$$

$$\dot{\vec{\omega}} = \mathbf{I}_b^{-1}\left(\vec{\tau}^b - \vec{\omega} \times \mathbf{I}_b\vec{\omega}\right) \tag{156}$$

The 2D case. In $2D$ only $\omega_z = \omega$ survives, and the gyroscopic term $\vec{\omega} \times \mathbf{I}_b\vec{\omega}$ vanishes entirely because there is only one rotation axis. The rotational equation reduces to the trivially simple $\dot{\omega} = \tau^b/J$. The full $2D$ (coupled) system in component-wise matrix form is:

$$\begin{pmatrix} \dot{v}_x \\ \dot{v}_y \\ \dot{\omega} \end{pmatrix} = \underbrace{\begin{pmatrix} 0 & +\omega & 0 \\ -\omega & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}}_{A(\omega)} \begin{pmatrix} v_x \\ v_y \\ \omega \end{pmatrix} + \begin{pmatrix} F_x^b/m \\ F_y^b/m \\ \tau^b/J \end{pmatrix} \tag{157}$$

Comparing with the uncoupled system derived above, the only difference is the $\pm\omega$ entries coupling v_x and v_y – as direct consequence of working in a rotating frame.

The 3D case. In $3D$ both coupling terms are present. For *principal body axes* (diagonal inertia tensor $\mathbf{I} = \mathrm{diag}(I_x, I_y, I_z)$), the full state-space system $\dot{\vec{u}} = A(\vec{\omega})\,\vec{u} + \vec{b}$ written out with all component equations is:

$$\begin{pmatrix} \dot{v}_x \\ \dot{v}_y \\ \dot{v}_z \\ \dot{\omega}_x \\ \dot{\omega}_y \\ \dot{\omega}_z \end{pmatrix} = \left(\begin{array}{ccc|ccc} 0 & +\omega_z & -\omega_y & 0 & 0 & 0 \\ -\omega_z & 0 & +\omega_x & 0 & 0 & 0 \\ +\omega_y & -\omega_x & 0 & 0 & 0 & 0 \\ \hline 0 & 0 & 0 & 0 & +\frac{I_y}{I_x}\omega_z & -\frac{I_z}{I_x}\omega_y \\ 0 & 0 & 0 & -\frac{I_x}{I_y}\omega_z & 0 & +\frac{I_z}{I_y}\omega_x \\ 0 & 0 & 0 & +\frac{I_x}{I_z}\omega_y & -\frac{I_y}{I_z}\omega_x & 0 \end{array}\right) \begin{pmatrix} v_x \\ v_y \\ v_z \\ \omega_x \\ \omega_y \\ \omega_z \end{pmatrix} + \begin{pmatrix} F_x^b/m \\ F_y^b/m \\ F_z^b/m \\ \tau_x^b/I_x \\ \tau_y^b/I_y \\ \tau_z^b/I_z \end{pmatrix} \tag{158}$$

The zeros in the off-diagonal 3×3 blocks show immediately that translational velocity components $\vec{v}$ and angular velocity components $\vec{\omega}$ do not appear directly in each other's rows – they are coupled only through the coefficients

ω_k that appear in the top-left block.

Block structure of $A(\vec{\omega})$. The same matrix has the compact block form

$$A(\vec{\omega}) = \begin{pmatrix} \underbrace{-[\vec{\omega}]_\times}_{\text{frame rotation}} & \mathbf{0} \\ \mathbf{0} & \underbrace{-\mathbf{I}_b^{-1}[\vec{\omega}]_\times \mathbf{I}_b}_{\text{gyroscopic}} \end{pmatrix} \tag{159}$$

where $[\vec{\omega}]_\times$ is the 3×3 antisymmetric cross-product matrix of $\vec{\omega}$. The role of each block is summarised in Table 10.

Block	Meaning
Top-left $-[\vec{\omega}]_\times$	v_x, v_y, v_z couple to each other – rotating framevdrags translational velocity
Bottom-right $-\mathbf{I}_b^{-1}[\vec{\omega}]_\times \mathbf{I}_b$	$\omega_x, \omega_y, \omega_z$ couple to each other – gyroscopic/Euler torque
Off-diagonal $\mathbf{0}$	No direct $v_i \leftrightarrow \omega_j$ coupling in the state

Table 10: Block structure of the coupling matrix $A(\vec{\omega})$ and the physical meaning of each block.

Several structural properties of this system are worth noting:

- The two subsystems ($\vec{v}$ and $\vec{\omega}$) *do not appear in each other's state slots* – the off-diagonal blocks are zero.
- The coupling is *parametric*: $\vec{\omega}$ appears as coefficients in the translational block, not as additional state variables.
- $\vec{\omega}$ drives $\vec{v}$ indirectly via the rotating frame; $\vec{v}$ does *not* influence $\dot{\vec{\omega}}$.
- The system is *nonlinear* because $A(\vec{\omega})$ itself depends on the state $\vec{\omega}$.
- Since $\vec{\omega}$ must be known to evaluate $A(\vec{\omega})$, both subsystems must be integrated together with a shared $\vec{\omega}$ at each timestep – they cannot be decoupled.

Reading row by row from $\dot{\vec{u}} = A(\vec{\omega})\vec{u} + \vec{b}$ yields the six coupled first-order

ODEs that are directly implementable in code:

$$\dot{v}_x = +\omega_z v_y - \omega_y v_z + F_x^b/m \tag{160a}$$
$$\dot{v}_y = -\omega_z v_x + \omega_x v_z + F_y^b/m \tag{160b}$$
$$\dot{v}_z = +\omega_y v_x - \omega_x v_y + F_z^b/m \tag{160c}$$
$$\dot{\omega}_x = \frac{I_y - I_z}{I_x}\,\omega_y\omega_z + \tau_x^b/I_x \tag{160d}$$
$$\dot{\omega}_y = \frac{I_z - I_x}{I_y}\,\omega_z\omega_x + \tau_y^b/I_y \tag{160e}$$
$$\dot{\omega}_z = \frac{I_x - I_y}{I_z}\,\omega_x\omega_y + \tau_z^b/I_z \tag{160f}$$

The last three equations (rotational) are the classical Euler equations for a rigid body with applied torque. Each contains a nonlinear product of two angular velocity components – the $\omega_i\omega_j$ terms are the gyroscopic contribution encoded in the bottom-right block of $A(\vec{\omega})$. The first three equations (translational) each couple two velocity components via ω_k, mirroring the $2D$ case but extended to all three axes.

Integrating orientation. To recover positions and orientations in the inertial frame, the orientation must be evolved alongside the dynamics. With a rotation matrix $\mathbf{R}$ and position $\vec{r}$, the kinematic equations are:

$$\dot{\vec{r}} = \mathbf{R}\,\vec{v}, \qquad \dot{\mathbf{R}} = \mathbf{R}\,[\vec{\omega}]_\times \tag{161}$$

In practice, quaternions $\mathbf{q}$ are strongly recommended over rotation matrices or Euler angles to avoid gimbal lock:

$$\dot{\mathbf{q}} = \tfrac{1}{2}\,\mathbf{q} \otimes \vec{\omega} \tag{162}$$

with the constraint $|\mathbf{q}| = 1$ enforced by periodic renormalisation.

Complete classical state vectors. The full state vector required for numerical integration – including position, orientation, and velocity degrees of freedom – is summarised in Tables 11 and 12.

State	DoF	Equation
x, y (inertial pos.)	2	$\dot{\vec{r}} = \mathbf{R}(\theta)\,\vec{v}$
θ (orientation)	1	$\dot{\theta} = \omega$
v_x, v_y (body vel.)	2	$\dot{v}_x = +\omega v_y + F_x^b/m, \quad \dot{v}_y = -\omega v_x + F_y^b/m$
ω (angular vel.)	1	$\dot{\omega} = \tau^b/J$
Total	6	coupled first-order ODEs

Table 11: Complete classical state vector for $2D$ rigid-body integration in the body frame.

State	DoF	Equation
$\vec{r}$ (inertial pos.)	3	$\dot{\vec{r}} = \mathbf{R}\,\vec{v}$
$\mathbf{q}$ (quaternion)	4	$\dot{\mathbf{q}} = \frac{1}{2}\mathbf{q} \otimes \vec{\omega}$
$\vec{v}$ (body vel.)	3	$\dot{\vec{v}} = \vec{F}^b/m - \vec{\omega} \times \vec{v}$
$\vec{\omega}$ (body ang. vel.)	3	$\dot{\vec{\omega}} = \mathbf{I}_b^{-1}\left(\vec{\tau}^b - \vec{\omega} \times \mathbf{I}_b\vec{\omega}\right)$
Total	13	coupled first-order ODEs (with $\lvert\mathbf{q}\rvert = 1$)

Table 12: Complete classical state vector for $3D$ rigid-body integration in the body frame.

Property	$2D$	$3D$
State size	6	13 (with quaternion)
Gyroscopic term $\vec{\omega} \times \mathbf{I}_b\vec{\omega}$	**Zero**	Present, nonlinear
Rotational dof	1 (ω scalar, decoupled)	3 ($\omega_x, \omega_y, \omega_z$ coupled)
Orientation representation	Scalar angle θ	Quaternion $\mathbf{q}$ (4 states)

Table 13: Comparison of $2D$ and $3D$ classical body-frame integration.

Switching to PGA: what changes, what unifies. We can now compare the complete classical body-frame system with the PGA formulation. The PGA equation of motion (142)

$$\dot{\Omega}_b = I_b^{-1}[F_b - \mathrm{rcmt}(\Omega_b, I_b[\Omega_b])]$$

looks like a single compact expression, but it encodes the same physics as both eqns. (155) and (156) together. Table 14 show the full correspondence. The key unification: where the classical formulation requires two separate differential equations (one for $\dot{\vec{v}}$ (3 DoF), one for $\dot{\vec{\omega}}$ (3 DoF)) plus a separate orientation update (quaternion or rotation matrix), the PGA ODE handles the first two in a single equation over the bivector $\dot{\Omega}_b$ (6 DoF), and by a separate update of the position using a motor directly derived from Ω_b. The rcmt term plays the combined role of both coupling terms of the classical approach ($\vec{\omega} \times \vec{v}$ and $\vec{\omega} \times \mathbf{I}_b\vec{\omega}$) in one operation.

Classical body frame	*PGA* equivalent
Two separate equations: Newton + Euler	One unified equation for $\dot{\Omega}_b$
Coupling term $\vec{\omega} \times \vec{v}$	Encoded in $\mathrm{rcmt}(\Omega_b, I_b[\Omega_b])$
Gyroscopic term $\vec{\omega} \times \mathbf{I}_b\vec{\omega}$	Also encoded in $\mathrm{rcmt}(\Omega_b, I_b[\Omega_b])$
Inertia tensor $\mathbf{I}_b$ (3×3 or 6×6 matrix)	Inertia map $I_b[\cdot]$ (linear operator on bivectors)
Quaternion evolution $\dot{\mathbf{q}} = \frac{1}{2}\mathbf{q} \otimes \vec{\omega}$	Motor $M = \exp_{\vee}[B_b]$ – orientation implicit in B_b
State: $(\vec{r}, \mathbf{q}, \vec{v}, \vec{\omega})$	State: (B_b, Ω_b) – bivector pair
Inertial-frame position from $\dot{\vec{r}} = \mathbf{R}\vec{v}$	Inertial-frame points updated via sandwich product $X(t) = M \vee X_b(t) \vee \underset{\sim}{M}$

Table 14: Classical body-frame vs. *PGA* formulation: explicit correspondence.

As just described, the *PGA* equation (142) does not directly contain the degrees of freedom we are ultimately interested in, like X, $\dot{X}$, and $\ddot{X}$. Instead, we first solve the system for the corresponding (bi)vectors[8] $B(t)$, $\dot{B} = \Omega(t)$, and $\ddot{B} = \dot{\Omega}(t)$, and then recover the actual points and their velocities in a follow-up step from the motor equations (68) and (69).

PGA state and initial conditions (2D case). To make these remarks concrete let's look into the $2D$ case. The position and rate of change vectors in the *body frame* are defined as

$$B_b = \begin{pmatrix} s_x \\ s_y \\ \phi \end{pmatrix}, \quad \dot{B}_b = \Omega_b = \begin{pmatrix} v_x \\ v_y \\ \omega \end{pmatrix}, \quad \dot{\Omega}_b = \begin{pmatrix} a_x \\ a_y \\ \alpha \end{pmatrix} \tag{163}$$

Comparing with Table 11, the *PGA* state $(B_{b,x}, \Omega_{b,x}, B_{b,y}, \Omega_{b,y}, B_{b,\phi}, \Omega_{b,\omega})$ corresponds directly to the classical state $(x, v_x, y, v_y, \theta, \omega)$ for the dynamic degrees of freedom. The inertial-frame positions of the center of mass (the origin of the body frame) and its orientation are recovered via the motor instead of via a separate kinematic integration. The full state vector transformed to 1st order is $u(B_b, \Omega_b) = (s_x, v_x, s_y, v_y, \phi, \omega)$. Suitable initial conditions for speed and angular speed can be directly derived from equations (112) and (113).

The initial value for the position is typically set to $B_b(t = 0) = (0, 0, 0)$, which yields the identity transformation at $t = 0$ as $M(t = 0) = \exp_{\vee}[B_b(t = 0)] = \mathbb{1}$.

[8] $2D$ case: vectors; $3D$ case: bivectors

This corresponds to the classical choice $\mathbf{q}(t=0) = (1,0,0,0)$ and $\vec{r}(t=0) = \vec{0}$ for the quaternion and position initial conditions.

Basic examples for translational and rotational motion are implemented as tests in `ga_test/src/ga_appl_physics_test.hpp` based on the Newton-Euler equations in both approaches, the classical approach and the approach directly formulated in PGA, for simple reference cases.

1.4.4 Structure of inertia and inertia maps

The inertia map $I[\Omega]$ derived in the preceding subsections is a linear map that takes the velocity twist Ω of a rigid body and returns its momentum wrench $P = I[\Omega]$. Its matrix representation – shown in eqns. (144) and (146) – encodes mass, scalar moment of inertia, and the full classical inertia tensor in a single algebraic object. However, the classical quantities do not sit where standard intuition might suggest: mass does not appear on the diagonal, and moments of inertia do not appear in the block connecting forces to accelerations of the same type.

The placement of each entry is determined by two ingredients: the choice of basis for the domain and codomain of the inertia map, and the complement structure of the respective algebra. The right complement $\overline{(\cdot)}$ maps each basis element to its complement in the algebra. In PGA the translational and rotational generators live in dual subspaces – ideal lines are dual to real lines – so the mass contribution, which couples velocity to momentum across dual subspaces, always lands in an off-diagonal block. The following paragraphs explain this structure first for $2D$, then for $3D$, and conclude with a unified comparison.

$PGA2DP$: structure of the 2D inertia map. In $2D$ PGA the velocity twist Ω is a vector and the momentum wrench P is a bivector, so the inertia map crosses two different grades:

$$I_{2D}[\Omega] : \text{Vec2dp} \to \text{BiVec2dp} \tag{164}$$

The basis elements and their physical roles are listed in Table 15.
For a unitized point (X_x, X_y, X_z) with projective coordinate $X_z = 1$ and mass m, the inertia matrix from eqn. (144) simplifies to

$$I_{2D} = m \begin{bmatrix} 0 & 1 & -X_y \\ -1 & 0 & X_x \\ -X_x & -X_y & X_x^2 + X_y^2 \end{bmatrix} \tag{165}$$

Index	Vec2dp input (velocity twist Ω)	BiVec2dp output (momentum wrench P)
0	$\boldsymbol{e}_1$ – x-velocity	$\boldsymbol{e}_{31}$ – x-momentum
1	$\boldsymbol{e}_2$ – y-velocity	$\boldsymbol{e}_{32}$ – y-momentum
2	$\boldsymbol{e}_3$ – angular velocity	$\boldsymbol{e}_{12}$ – angular momentum

Table 15: Basis correspondence for the $PGA2DP$ inertia map.

where rows correspond to output components ($\boldsymbol{e}_{31}$, $\boldsymbol{e}_{32}$, $\boldsymbol{e}_{12}$) and columns to input components (Ω_x, Ω_y, Ω_z).

Why mass is off-diagonal. The mass contribution to the inertia map takes the form $I_{\text{mass}}(\Omega) = -m \cdot \overline{\Omega}$, where $\overline{(\cdot)}$ denotes the complement in $PGA2DP$. The complement of the translational basis vectors crosses indices:

$$\overline{\boldsymbol{e}_1} = \boldsymbol{e}_{32}, \qquad \overline{\boldsymbol{e}_2} = -\boldsymbol{e}_{31} \tag{166}$$

Applying $I_{\text{mass}} = -m \cdot \overline{(\cdot)}$ gives the following contributions:

- Ω_x (component along $\boldsymbol{e}_1$, column 0) maps to $-m \cdot \boldsymbol{e}_{32}$, landing at output row 1 $\rightarrow I[1,0] = -m$.
- Ω_y (component along $\boldsymbol{e}_2$, column 1) maps to $-m \cdot (-\boldsymbol{e}_{31}) = +m \cdot \boldsymbol{e}_{31}$, landing at output row 0 $\rightarrow I[0,1] = +m$.

The diagonal entries $I[0,0]$ and $I[1,1]$ are zero because the complement of $\boldsymbol{e}_1$ is $\boldsymbol{e}_{32}$ (not $\boldsymbol{e}_{31}$) and the complement of $\boldsymbol{e}_2$ is $-\boldsymbol{e}_{31}$ (not $\boldsymbol{e}_{32}$): there is no self-coupling. This is a direct consequence of the degenerate metric $\boldsymbol{e}_3^2 = 0$.

The resulting mass block is antisymmetric with $I[0,1] = +m$ and $I[1,0] = -m$. The sign structure is consistent with the kinematic encoding: the motor-generator Ω for a pure translation with classical velocity (v_x, v_y) is encoded in $PGA2DP$ as $\Omega = (-v_y, v_x, 0)$. The two sign flips cancel and the classical linear momentum $p_x = mv_x$, $p_y = mv_y$ is correctly recovered. In any other context – position, force, acceleration – $\boldsymbol{e}_1$ always encodes the x-component and $\boldsymbol{e}_2$ the y-component.

Moment of inertia and cross-coupling terms. The entry $I[2,2] = m(X_x^2 + X_y^2) = mr^2$ maps angular velocity ($\boldsymbol{e}_3$, index 2) to angular momentum ($\boldsymbol{e}_{12}$, index 2), recovering the classical scalar moment of inertia $J = mr^2$ for a point mass at distance r from the rotation center. The complement of $\boldsymbol{e}_3$ is $\overline{\boldsymbol{e}_3} = -\boldsymbol{e}_{12}$, which connects the angular velocity generator directly to the

angular momentum carrier; the sign is absorbed into the definition so that $J > 0$. The entries $I[0,2] = -mX_y$ and $I[1,2] = +mX_x$ couple angular velocity to linear momentum, encoding the classical relation $\vec{L}_{\text{linear}} = m(\vec{\omega} \times \vec{r})$ for a body rotating at offset (X_x, X_y) from the rotation center. These entries, together with their transposes $I[2,0]$ and $I[2,1]$, vanish when the point is at the origin.

Body-frame inertia matrix. Integrating the single-point formula over a rigid body with center of mass at the origin (body frame) gives:

$$I_{\text{body}} = \begin{bmatrix} 0 & m & 0 \\ -m & 0 & 0 \\ 0 & 0 & J \end{bmatrix} \tag{167}$$

where $J = \int_m (x^2 + y^2)\, dm$ is the classical scalar moment of inertia about the center of mass. The antisymmetric mass block (upper-left 2×2) and the moment of inertia entry (lower-right 1×1) are cleanly separated: translational and rotational degrees of freedom decouple at the center of mass, exactly as in classical mechanics.

General frame with center of mass at (c_x, c_y). When the chosen reference point does not coincide with the center of mass, the integrated inertia matrix is:

$$I = \begin{bmatrix} 0 & m & -mc_y \\ -m & 0 & mc_x \\ -mc_x & -mc_y & J_{\text{cm}} + m(c_x^2 + c_y^2) \end{bmatrix} \tag{168}$$

where J_{cm} is the scalar moment of inertia about the center of mass and $r^2 = c_x^2 + c_y^2$ is the squared distance from the center of mass to the reference point. The matrix decomposes into the following sub-blocks:

- *Upper-left* 2×2 $[0, m;\ -m, 0]$: the antisymmetric mass block. It is *frame-invariant* – its values are unchanged regardless of where the center of mass is located.
- *Upper-right* 2×1 $[-mc_y;\ mc_x]$: angular velocity → linear momentum coupling. Encodes $\vec{L}_{\text{linear}} = m(\vec{\omega} \times \vec{r}_{\text{cm}})$. Zero only when the center of mass coincides with the reference point.
- *Lower-left* 1×2 $[-mc_x,\ -mc_y]$: translational velocity → angular momentum coupling. Transpose of the upper-right block (up to sign convention).

- *Lower-right* 1×1: $J_{\text{cm}} + m(c_x^2 + c_y^2)$ – the classical parallel axis theorem applied to the scalar moment of inertia.

Continuous limit: uniform rectangular plate. As a verification, integrating over a uniform plate of width W (along $\boldsymbol{e}_1$), height H (along $\boldsymbol{e}_2$), total mass M, centerd at the origin gives:

$$I_{\text{plate}} = M \begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & \dfrac{W^2 + H^2}{12} \end{bmatrix} \tag{169}$$

The entry $I[2,2] = M(W^2 + H^2)/12$ is the standard formula for the moment of inertia of a uniform rectangle about its centroidal axis. For a discrete endpoint-inclusive grid of $n \times n$ equally spaced point masses, the relative error in $I[2,2]$ compared to the continuous value is $2/(n-1)$, independent of plate dimensions, converging from above as $n \to \infty$.

Table 16 summarises the classical inertial quantities and their locations in the *PGA2DP* inertia matrix.

Classical quantity	*PGA2DP* entry	Condition
Total mass m	$I[0,1] = +m,\ \ I[1,0] = -m$	Always
Scalar moment J about CoM	$I[2,2] = J$	CoM at origin
CoM offset (c_x, c_y)	$I[0,2] = -mc_y,\ \ I[1,2] = +mc_x$ (and transposed)	General frame
Parallel axis correction	$I[2,2] = J_{\text{cm}} + m(c_x^2 + c_y^2)$	General frame

Table 16: Classical inertial quantities and their location in the *PGA2DP* 3×3 inertia matrix.

PGA3DP: structure of the 3D inertia map. In 3*D PGA* both the velocity twist Ω and the momentum wrench P are bivectors, so the inertia map acts within the same grade:

$$I_{3D}[\Omega] : \text{BiVec3dp} \to \text{BiVec3dp} \tag{170}$$

The six basis bivectors split into two groups: *ideal lines* ($\boldsymbol{e}_{41}$, $\boldsymbol{e}_{42}$, $\boldsymbol{e}_{43}$, containing the projective direction $\boldsymbol{e}_4$ which squares to zero) carrying translational information, and *real lines* ($\boldsymbol{e}_{23}$, $\boldsymbol{e}_{31}$, $\boldsymbol{e}_{12}$) of the Euclidean subspace carrying rotational information. Their physical roles are listed in Table 17.

Idx	BiVec3dp input (velocity twist Ω)	BiVec3dp output (momentum wrench P)
0	$\boldsymbol{e}_{41}$ – ideal line, x-velocity	$\boldsymbol{e}_{41}$ – linear momentum x
1	$\boldsymbol{e}_{42}$ – ideal line, y-velocity	$\boldsymbol{e}_{42}$ – linear momentum y
2	$\boldsymbol{e}_{43}$ – ideal line, z-velocity	$\boldsymbol{e}_{43}$ – linear momentum z
3	$\boldsymbol{e}_{23}$ – real line, angular velocity about $\boldsymbol{e}_1$	$\boldsymbol{e}_{23}$ – angular momentum about $\boldsymbol{e}_1$
4	$\boldsymbol{e}_{31}$ – real line, angular velocity about $\boldsymbol{e}_2$	$\boldsymbol{e}_{31}$ – angular momentum about $\boldsymbol{e}_2$
5	$\boldsymbol{e}_{12}$ – real line, angular velocity about $\boldsymbol{e}_3$	$\boldsymbol{e}_{12}$ – angular momentum about $\boldsymbol{e}_3$

Table 17: Basis correspondence for the $PGA3DP$ inertia map.

The 6×6 inertia matrix from eqn. (146) for a unitized point with $X_w = 1$ is:

$$I_{3D} = m \begin{bmatrix} 0 & X_z & -X_y & 1 & 0 & 0 \\ -X_z & 0 & X_x & 0 & 1 & 0 \\ X_y & -X_x & 0 & 0 & 0 & 1 \\ X_y^2 + X_z^2 & -X_x X_y & -X_x X_z & 0 & -X_z & X_y \\ -X_x X_y & X_x^2 + X_z^2 & -X_y X_z & X_z & 0 & -X_x \\ -X_x X_z & -X_y X_z & X_x^2 + X_y^2 & -X_y & X_x & 0 \end{bmatrix} \tag{171}$$

where columns correspond to (Ω_{vx}, Ω_{vy}, Ω_{vz}, Ω_{mx}, Ω_{my}, Ω_{mz}) and rows to the same ordering for P.

Block structure and the complement in 3D. The 6×6 matrix decomposes into four 3×3 blocks with distinct physical roles:

- *Upper-left [0:3, 0:3]*: translational velocity → linear momentum. For a point at the origin this block is zero. For a point displaced to (X_x, X_y, X_z) it becomes the antisymmetric cross-product matrix $[X\times]$, encoding the linear momentum contribution due to the off-axis position.

- *Upper-right [0:3, 3:6]*: angular velocity → linear momentum. For a unitized point ($X_w = 1$) this block equals $m\mathbf{I}_3$ (mass times the 3×3 identity). This is the $3D$ analogue of the off-diagonal mass entries in the $2D$ case.

- *Lower-left [3:6, 0:3]*: translational velocity → angular momentum. This block carries the classical moments and products of inertia.

- *Lower-right [3:6, 3:6]*: angular velocity → angular momentum. Zero for a point at the origin; becomes antisymmetric for a displaced center of mass.

The placement of mass in the upper-right block follows from the complement in $PGA3DP$, which maps real lines to ideal lines:

$$\overline{\boldsymbol{e}_{23}} = \boldsymbol{e}_{41}, \quad \overline{\boldsymbol{e}_{31}} = \boldsymbol{e}_{42}, \quad \overline{\boldsymbol{e}_{12}} = \boldsymbol{e}_{43} \tag{172}$$

Applying $I_{\text{mass}} = -m \cdot \overline{(\cdot)}$ maps each angular velocity component to the corresponding linear momentum component, placing m on the diagonal of the upper-right block: $I[0,3] = I[1,4] = I[2,5] = m$. The diagonal entries of the upper-left and lower-right blocks are zero by the same degenerate-metric argument as in $2D$.

Body-frame inertia matrix. Integrating the single-point formula over a rigid body with center of mass at the origin gives:

$$I_{\text{body}} = \begin{bmatrix} 0 & 0 & 0 & m & 0 & 0 \\ 0 & 0 & 0 & 0 & m & 0 \\ 0 & 0 & 0 & 0 & 0 & m \\ I_{xx} & -I_{xy} & -I_{xz} & 0 & 0 & 0 \\ -I_{xy} & I_{yy} & -I_{yz} & 0 & 0 & 0 \\ -I_{xz} & -I_{yz} & I_{zz} & 0 & 0 & 0 \end{bmatrix} \tag{173}$$

written in block form as $I_{\text{body}} = [\mathbf{0},\ m\mathbf{I}_3;\ \mathbf{J}_{cm},\ \mathbf{0}]$, where $\mathbf{J}_{cm}$ is the classical 3×3 inertia tensor about the center of mass:

$$\mathbf{J}_{cm} = \begin{bmatrix} I_{xx} & -I_{xy} & -I_{xz} \\ -I_{xy} & I_{yy} & -I_{yz} \\ -I_{xz} & -I_{yz} & I_{zz} \end{bmatrix} \tag{174}$$

with the classical integrals for the diagonal moments $I_{xx} = \int_m (y^2 + z^2)\, dm$, $I_{yy} = \int_m (x^2 + z^2)\, dm$, $I_{zz} = \int_m (x^2 + y^2)\, dm$ and the off-diagonal products of inertia $I_{xy} = \int_m xy\, dm$, $I_{xz} = \int_m xz\, dm$, $I_{yz} = \int_m yz\, dm$. Translational and rotational degrees of freedom decouple at the center of mass: the diagonal blocks are zero and all physical content sits in the off-diagonal blocks.

General frame with center of mass at $\vec{r}_{cm} = (c_x, c_y, c_z)$. In a general frame all

four 3×3 blocks are non-zero:

$$I = \begin{bmatrix} 0 & mc_z & -mc_y & m & 0 & 0 \\ -mc_z & 0 & mc_x & 0 & m & 0 \\ mc_y & -mc_x & 0 & 0 & 0 & m \\ I_{xx} & -I_{xy} & -I_{xz} & 0 & -mc_z & mc_y \\ -I_{xy} & I_{yy} & -I_{yz} & mc_z & 0 & -mc_x \\ -I_{xz} & -I_{yz} & I_{zz} & -mc_y & mc_x & 0 \end{bmatrix} \tag{175}$$

where the lower-left inertia tensor entries include the classical parallel axis corrections:

$$\begin{aligned} I_{xx} &= I_{xx,cm} + m(c_y^2 + c_z^2), & I_{xy} &= I_{xy,cm} + mc_x c_y \\ I_{yy} &= I_{yy,cm} + m(c_x^2 + c_z^2), & I_{xz} &= I_{xz,cm} + mc_x c_z \\ I_{zz} &= I_{zz,cm} + m(c_x^2 + c_y^2), & I_{yz} &= I_{yz,cm} + mc_y c_z \end{aligned} \tag{176}$$

Two structural features carry over from the $2D$ case:

- *The upper-right block $m\,\mathbf{I}_3$ is frame-invariant.* It remains equal to $m\,\mathbf{I}_3$ regardless of the center-of-mass position. Mass is a dual coupling between ideal and real lines that does not depend on position. This is the $3D$ counterpart of the invariant antisymmetric mass block in $2D$.

- *The upper-left and lower-right blocks are conjugate antisymmetric matrices.* Upper-left = $m[\vec{r}_{cm}\times]^\top$ and lower-right = $m[\vec{r}_{cm}\times]$, where $[\vec{r}_{cm}\times]$ is the 3×3 antisymmetric cross-product matrix of the center-of-mass position vector. Both blocks are driven by the same offset $\vec{r}_{cm}$, reflecting that the translational-to-translational and angular-to-angular couplings induced by the offset are conjugate couplings.

This 6×6 structure is the PGA form of the classical spatial inertia matrix from rigid body dynamics [Dorst and De Keninck, 2023], expressed in the (translational, angular) velocity ordering of $PGA3DP$.

Continuous limit: uniform rectangular cuboid. For a uniform cuboid of width w (along $\boldsymbol{e}_1$), height h (along $\boldsymbol{e}_2$), depth d (along $\boldsymbol{e}_3$), total mass m, centerd

at the origin, all cross-inertia and center-of-mass coupling terms vanish:

$$I_{\text{cuboid}} = m \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ \frac{h^2+d^2}{12} & 0 & 0 & 0 & 0 & 0 \\ 0 & \frac{w^2+d^2}{12} & 0 & 0 & 0 & 0 \\ 0 & 0 & \frac{w^2+h^2}{12} & 0 & 0 & 0 \end{bmatrix} \tag{177}$$

The three diagonal entries of the lower-left block are the classical rectangular moments of inertia: $I[3,0] = m(h^2+d^2)/12$ about the x-axis, $I[4,1] = m(w^2+d^2)/12$ about the y-axis, and $I[5,2] = m(w^2+h^2)/12$ about the z-axis – each determined by the two extents of the body perpendicular to the respective rotation axis, as in the classical formula.

Table 18 gives the correspondence between classical inertial quantities and their location in the *PGA3DP* inertia matrix.

Classical quantity	*PGA3DP* block	Entries (body frame, CoM at origin)
Total mass m	Upper-right diagonal ($m\mathbf{I}_3$)	$I[0,3] = I[1,4] = I[2,5] = m$
Diagonal moments I_{xx}, I_{yy}, I_{zz}	Lower-left diagonal	$I[3,0],\ I[4,1],\ I[5,2]$
Products of inertia I_{xy}, I_{xz}, I_{yz}	Lower-left off-diagonal	$I[3,1] = I[4,0] = -I_{xy}$, etc.
CoM offset (general frame)	All four 3×3 blocks	via parallel axis theorem

Table 18: Classical inertial quantities and their location in the *PGA3DP* 6×6 inertia matrix (body frame, center of mass at origin).

Unified view: 2D and 3D. In both *PGA2DP* and *PGA3DP* the mass contribution to the inertia map takes the same algebraic form:

$$I_{\text{mass}}(\Omega) = -m \cdot \overline{\Omega} \tag{178}$$

The complement maps each velocity generator to its momentum carrier in the dual subspace, placing mass in the off-diagonal block that connects dual velocity and momentum spaces. The location of that block in the matrix

depends on the basis ordering.

In $PGA2DP$ the translational generators ($\boldsymbol{e}_1$, $\boldsymbol{e}_2$, indices 0–1) sit at lower indices and the rotational generator ($\boldsymbol{e}_3$, index 2) sits at the highest index. The complement maps translational inputs to translational outputs while crossing x and y, placing mass in the *upper-left* 2×2 sub-block (translational velocity → linear momentum).

In $PGA3DP$ the translational generators ($\boldsymbol{e}_{41}$, $\boldsymbol{e}_{42}$, $\boldsymbol{e}_{43}$, indices 0–2) are ideal lines at lower indices, while the rotational generators ($\boldsymbol{e}_{23}$, $\boldsymbol{e}_{31}$, $\boldsymbol{e}_{12}$, indices 3–5) are real lines at higher indices. The complement maps real-line inputs (angular velocity) to ideal-line outputs (linear momentum), placing mass in the *upper-right* 3×3 sub-block (angular velocity → linear momentum).

Table 19 gives the side-by-side comparison for the body frame (center of mass at origin).

Classical quantity	$PGA2DP$ (3×3)	$PGA3DP$ (6×6)
Total mass m	Upper-left 2×2: $I[0,1] = +m$, $I[1,0] = -m$	Upper-right 3×3 diag.: $I[0,3] = I[1,4] = I[2,5] = m$
Scalar/diagonal moments	Lower-right 1×1: $I[2,2] = J$	Lower-left 3×3 diag.: $I[3,0], I[4,1], I[5,2]$
Products of inertia	n/a (scalar J in $2D$)	Lower-left 3×3 off-diagonal
CoM coupling	$I[0,2], I[1,2]$ and transposed	All four 3×3 blocks non-zero
Parallel axis	$I[2,2] = J_{\mathrm{cm}} + mr^2$	Lower-left 3×3 (3D formula)

Table 19: Unified mapping of classical inertial quantities to the PGA inertia matrix (body frame, center of mass at origin).

The frame-invariance of the mass block and the decoupling at the center of mass are common to both dimensions. In the body frame with the origin at the center of mass – the natural choice for integrating eqn. (142) – the coupling blocks vanish, the mass block takes its simplest form, and the moment block reduces to J (in $2D$) or $\mathbf{J}_{cm}$ (in $3D$) without any parallel axis corrections. This matches the classical result that Newton's and Euler's equations decouple at the center of mass, and it is the reason the body frame with the center of mass at the origin is used as the preferred reference frame throughout this section. Note, however, that while the inertia map I_{body} is block-diagonal at the center of mass – translational and rotational blocks decoupling in the inertia matrix itself – the equations of motion retain dynamical coupling through the $\mathrm{rcmt}(\Omega_b, I_b[\Omega_b])$ term. This is the PGA

counterpart of the classical coupling terms $\vec{\omega} \times \vec{v}$ and $\vec{\omega} \times \mathbf{I}\vec{\omega}$ discussed in the preceding section, and it arises from the rotation of the body frame, not from the inertia structure.

1.5 Approaches with increasing expressivenes

1.5.1 Classic approach (vector algebra)

- Use of scalars and vectors, exclusively
- Use of the cross-product between vectors to create directions perpendicular to both vectors (a concept that can only be defined in three-dimensional space)
- Rotations are defined by a rotation axis (a concept that is only defined in three-dimensional space)[9]

1.5.2 General observations for Geometric Algebra (GA)

- The GA approach is coordinate-free!
- W.r.t. a vector, the dot product (or interior product) takes into account the parts parallel to the vector, exclusively. Thus, it is not invertible itself.
- W.r.t. a vector, the wedge product (or outer product) takes into account the parts perpendicular to the vector, exclusively. Thus, it is not invertible itself.
- Only the geometric product taking both parts into account is invertible.

1.5.3 Euclidean Geometric Algebra (EGA)

- Use of scalars, vectors, bivectors, ..., pseudoscalars (higher grade objects up to order n depend on the dimension n of the modelled space).
- Use of the outer product (wedge product) between vectors, which defines the space spanned by both vectors (can be defined in any dimension, used as join operation).
- Use of the regressive outer product (regressive wedge product) which defines the space shared by both objects (represents the intersection of the spaces used by the objects, used as meet operation).
- Modelled objects cover scalars, directions, planes (in $3D$) and the whole space directly.

[9] the last two concepts cannot be extended to higher-dimensional spaces

- All objects modelled in the algebra contain the (non-distinguishable) origin $O = (0, 0, ..., 0)$. Objects outside of the origin must be modeled by explicitly adding translations (leading to objects that cannot directly be manipulated as easily as objects that directly represent subspaces within the algebra).
- Orthogonal transformations in the algebra can directly model reflections and rotations. Rotations are defined in a plane spanned by two vectors defining the rotation.

1.5.4 Projective (Euclidean) Geometric Algebra (PGA)

- Like EGA, but in addition having a distinguishable origin encoded in an additional dimension $O = (0, 0, ..., 0, 1)$ (in a projective space with an additional dimension projected onto a fixed value, typically 1).
- Use of the outer and regressive outer products as join and meet operations.
- Directly models points, directions and planes (in $3D$), as well as the whole space, including objects located in the horizon, i.e. at infinity (points at infinity are modeled as directions towards those points).
- Can directly model higher grade objects by products between objects in the algebra that do not contain the origin, e.g. the wedge product between two arbitrary points defines a line.
- Orthogonal transformations in the algebra can directly model reflections, rotations and translations.

1.6 Glossary

Moment and torque (source wikipedia):

- A moment is a mathematical expression involving the product of a distance and a physical quantity such as a force or electric charge. Moments are usually defined with respect to a fixed reference point and refer to physical quantities located some distance from the reference point. For example, the moment of force, often called torque, is the product of a force on an object and the distance from the reference point to the object. In principle, any physical quantity can be multiplied by a distance to produce a moment.

- In physics and mechanics, torque is the rotational correspondent of linear force. It is also referred to as the moment of force, or simply the moment. Just as a linear force is a push or a pull applied to a body, a torque can be thought of as a twist applied to an object with respect to a chosen point.

- Torque is generally referred to using different vocabulary depending on geographical location and field of study. Torque and moment are often used interchangeably for the rotational effect of a force, but in mechanics, the main difference is application: torque refers to a force causing rotation in a dynamic system, while moment is often used for the turning effect in a static, or non-moving, system. Both are calculated as the force multiplied by the perpendicular distance from the pivot point.

- When modelling physics in PGA forces and moments, describing the translational and rotational physical aspects, do not need to be considered separately, but can be modelled within one object containing all information. For example, by modelling the line of action F of a force f acting at a point P as $F = P \wedge f$ as in equation (114) both effects are covered, the force f itself and the torque generated at any point are covered by the mathematical object F. The torque generated at the origin $M_O = bulk(F)$ can be as easily recovered from F as the resulting free moment at any other point.

1.7 Links to get started

Overview to easily get started (high-level motivation and initial introductions on youtube):

- *https://www.thestrangeloop.com* (Jack Rusher, Funny intro to GA with historical perspective)
- *https://www.youtube.com/@sudgylacmoe* (sudgylacmoe, A swift intro to GA)
- *https://www.youtube.com/@bivector/GAME23* (De Keninck and Roelfs, GAME 2023 - GA Intro)

Sources for GA and PGA as a mathematical topic:

- *https://projectivegeometricalgebra.org* (Eric Lengyel)
- *https://terathon.com/blog/poor-foundations-ga.html* (Eric Lengyel)
- *https://bivector.net* (Leo Dorst et.al.)
- *https://www.youtube.com/@bivector* (Youtube channel bivector)
- *https://en.wikipedia.org/wiki/Bivector* (Wikipedia Bivector)
- *https://www.youtube.com/@sudgylacmoe* (Youtube channel sudgylacmoe)
- *https://www.youtube.com/@hestenes* (Hestenes, Tutorial Geometric Calculus (relevant formulas))
- *https://en.wikipedia.org/PGA* (Plane-based GA in $R^*(3,0,1)$ instead of $R(3,0,1)$ as used here)

Sources for GA and PGA in more specific applications:

- *https://www.youtube.com/DynamicsPGA* (Leo Dorst, Dynamics in PGA)
- *https://en.wikipedia.org/wiki/Newton-Euler* (Newton-Euler equations)
- *https://en.wikipedia.org/wiki/Rigidbody* (Rigid body dynamics)
- *https://en.wikipedia.org/wiki/Inertia* (Moment of inertia)
- *https://en.wikipedia.org/wiki/ScrewTheory* (Screw theory, twist and wrench)
- https://en.wikipedia.org/wiki/Chasles (Chasles' theorem)
- http://web.cs.iastate.edu/Pluecker (Plücker coordinates in projective spaces)

1.8 Literature

References

[Browne, 2012] Browne, J. (2012). *Grassmann Algebra*, volume 1: Foundations. Barnard Publishing.

[Dorst and De Keninck, 2023] Dorst, L. and De Keninck, S. (August 15, 2023). May the forque be with you - dynamics in pga. *University of Amsterdam, The Netherlands*, V2.6.

[Dorst and De Keninck, 2022] Dorst, L. and De Keninck, S. (March 14, 2022). A guided tour to the plane-based geometric algebra pga. *University of Amsterdam, The Netherlands*, V2.0.

[Gunn, 2020] Gunn, C. G. (August 2020). Projective geometric algebra: A new framework for doing euclidean geometry. *Raum+Gegenraum Falkensee, Germany*.

[Hestenes, 1986] Hestenes, D. (1986). *New foundations for classical mechanics*. D. Reidel publishing company.

[Kritchevsky, 2024] Kritchevsky, A. (February 28, 2024). The case against geometric algebra. *Alex Kritchevsky's blog*.

[Lengyel, 2024b] Lengyel, E. (2024b). *Projective Geometric Algebra Illuminated*. Terathon Software LLC.

[Lengyel, 2024a] Lengyel, E. (August 23, 2024a). Poor Foundations in Geometric Algebra. *Eric Lengyel's blog*.

[Lengyel, 2025] Lengyel, E. (May 23, 2025). The Transwedge Product. *Eric Lengyel's blog*.